

# GEOMETRY

---

**Lecturer: Dr. Ufuk Çelikcan**

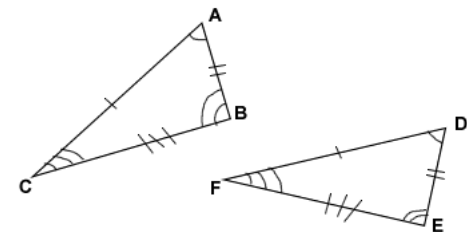
Based on the slides by: E. Angel and D. Shreiner

# Basic Elements

- **Geometry** is the study of the relationships among objects in an  $n$ -dimensional space
  - In computer graphics, we are interested in objects that exist in 3 dimensions
- Want a minimum set of primitives from which we can build more sophisticated objects
- We will need three basic elements
  1. **Scalars**
  2. **Vectors**
  3. **Points**

# Coordinate-Free Geometry

- When we learned simple geometry, most of us started with a Cartesian approach
  - Points at locations in space  $\mathbf{p}=(x,y,z)$
  - We derived results by algebraic manipulations involving these coordinates
- This approach is non-physical
  - Physically, points exist regardless of the location of an arbitrary coordinate system
  - Most geometric results are independent of the coordinate system
    - *Example:* Euclidean geometry: two triangles are identical if two corresponding sides and the angle between identical

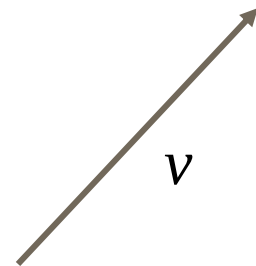


# Scalars

- Scalars can be defined as members of sets
  - which can be combined by two operations: addition and multiplication
  - obeying some fundamental axioms: associativity, commutativity, inverses
- Examples include the real and complex number systems under the ordinary rules with which we are familiar.
- Scalars alone have no geometric properties

# Vectors

- Physical definition:  
a vector is a quantity with two attributes
  - **Direction**
  - **Magnitude**
- Examples include
  - Force
  - Velocity
  - Directed line segments
    - Most important example for graphics
    - Can map to other types





# Scalar vs Vector

## SCALAR

A scalar quantity has magnitude only.



speed



mass



volume



time

## VECTOR

A vector has both magnitude and direction.



velocity



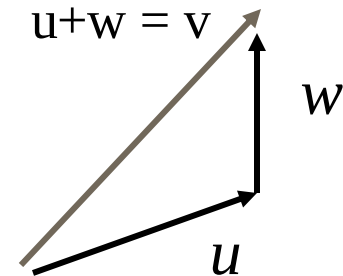
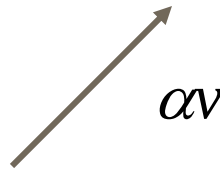
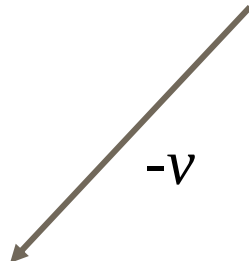
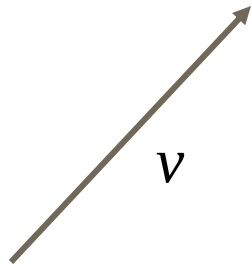
weight

friction



# Vector Operations

- Every vector has an inverse
  - Inverse has same magnitude but in opposite direction
- Every vector can be multiplied by a scalar
- There is a zero vector
  - Zero magnitude, undefined orientation
- The sum of any two vectors is a vector
  - Use head-to-tail axiom

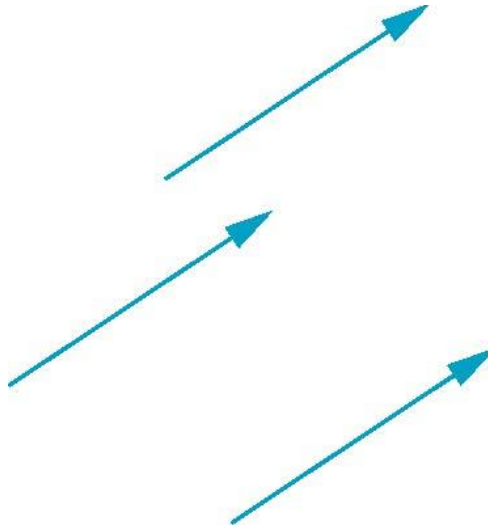


# Linear Vector Spaces

- Mathematical system for manipulating vectors
- Operations
  - scalar-vector multiplication  $u = \alpha v$
  - vector-vector addition:  $w = u + v$
- Expressions such as
$$v = u + 2w - 3r$$
make sense in a vector space

# Vectors Lack Position

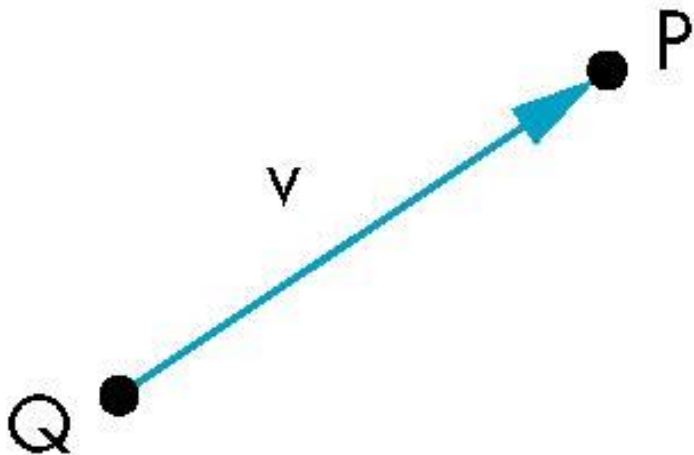
- These vectors are identical
  - Same length and magnitude



- >> Vector spaces are insufficient for geometry
  - >> Need to have points too

# Points

- Location in space
- Operations allowed between points and vectors
  - Point-point subtraction  $\gg$  yields a vector
    - Equivalent to point-vector addition



$$v = P - Q$$

$$P = v + Q$$

# Affine Spaces

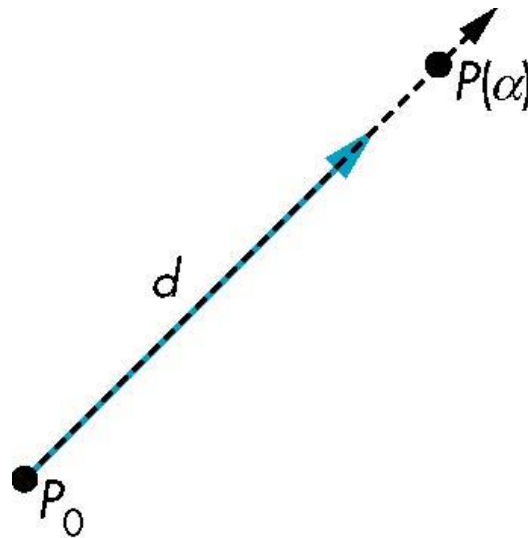
- Points + a vector space
  - Points are typically used to position things(3D models etc.) in a defined space
  - Vectors are used to move things in that space
- Operations
  - Vector-vector addition
  - Scalar-vector multiplication
  - Point-vector addition
  - Scalar-scalar operations
- For any point  $P$  define
  - $1 \bullet P = P$
  - $0 \bullet P = \mathbf{0}$  (zero vector)

# Lines

- Consider all points of the form

$$P(\alpha) = P_0 + \alpha \mathbf{d}$$

>> Set of all points that pass through  $P_0$  in the direction of the vector  $\mathbf{d}$



# Parametric Form

- Two-dimensional forms

1. Explicit form:  $y = mx + h$
2. Implicit form:  $ax + by + c = 0$

3. **Parametric form:**

$$x(\alpha) = \alpha x_0 + (1-\alpha)x_1$$

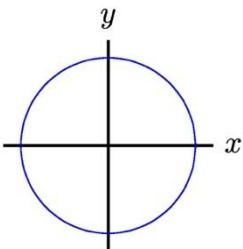
$$y(\alpha) = \alpha y_0 + (1-\alpha)y_1$$

- parametric form of the line**

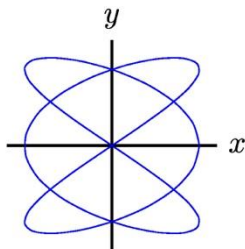
- More robust and general than other forms
- Extends to curves and surfaces



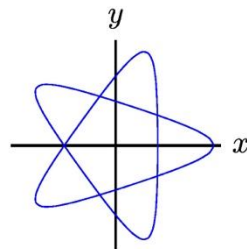
parametric curves:



$$\begin{aligned} x(t) &= \cos(t) \\ y(t) &= \sin(t) \end{aligned}$$



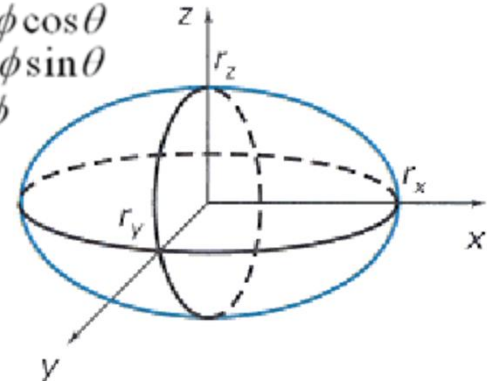
$$\begin{aligned} x(t) &= 3 \cos(t) \\ y(t) &= 2 \sin(t) \end{aligned}$$



$$\begin{aligned} x(t) &= 2 \cos(x) + 5 \cos\left(\frac{2}{3}x\right) \\ y(t) &= 2 \sin(x) + 5 \sin\left(\frac{2}{3}x\right) \end{aligned}$$

**parametric surface:  
ellipsoid**

$$\begin{aligned} x &= r_x \cos \phi \cos \theta \\ y &= r_y \cos \phi \sin \theta \\ z &= r_z \sin \phi \end{aligned}$$



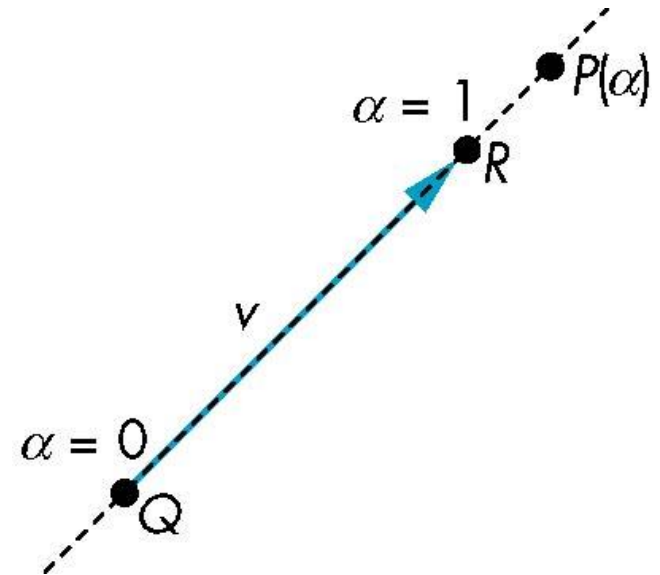
# Rays and Line Segments

- If  $\alpha \geq 0$ , then  $P(\alpha)$  is the **ray** leaving  $P_0$  in the direction  $\mathbf{d}$

If we use two points to define  $\mathbf{v}$ , then

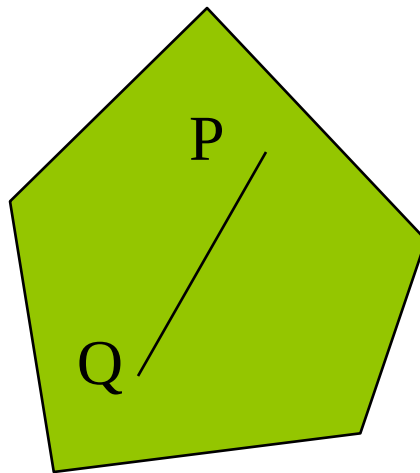
$$\begin{aligned} P(\alpha) &= Q + \alpha (R - Q) = Q + \alpha \mathbf{v} \\ &= \alpha R + (1 - \alpha)Q \end{aligned}$$

For  $0 \leq \alpha \leq 1$  we get all the points on the **line segment** joining R and Q

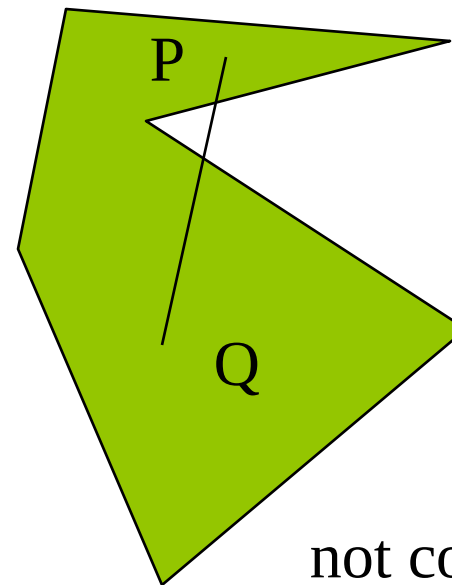


# Convexity

- An object is *convex* iff (*if and only if*) for any 2 points in the object all points on the line segment between these 2 points are also in the object



convex



not convex

# Affine Sums

For a given point  $P$  and an arbitrary set of points  $\{P_1, P_2, \dots, P_n\}$ , consider the sum

$$P = \alpha_1 P_1 + \alpha_2 P_2 + \dots + \alpha_n P_n$$

If  $\alpha_1 + \alpha_2 + \dots + \alpha_n = 1$

then we have the ***affine sum (affine combination)*** of the points  $P_1, P_2, \dots, P_n$

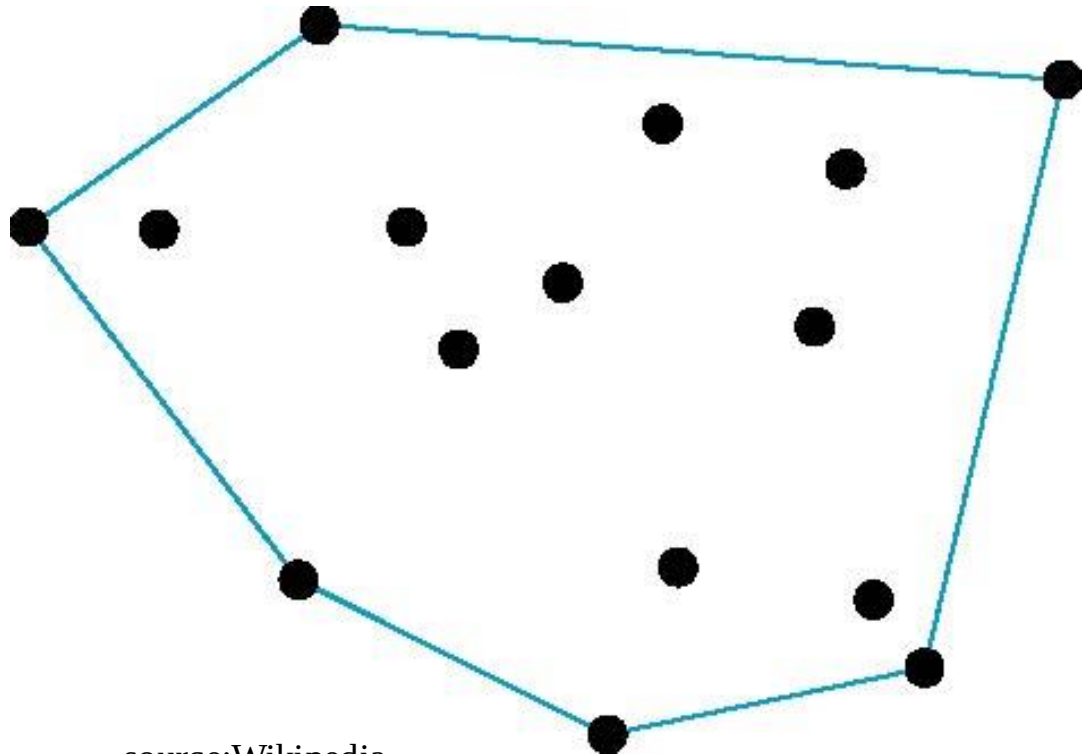
If also  $\alpha_i \geq 0$ , then we have the ***convex hull*** of  $P_1, P_2, \dots, P_n$

- Convex combinations are simply affine combinations where the constants in the combination are limited to be in the interval  $[0,1]$ .

# Convex Hull

- Smallest convex object containing  $P_1, P_2, \dots, P_n$ 
  - The set of all points  $P$  that can be written as convex combinations of  $P_1, P_2, \dots, P_n$
- Formed by “shrink wrapping” points

$$P = \alpha_1 P_1 + \alpha_2 P_2 + \dots + \alpha_n P_n$$



source:Wikipedia

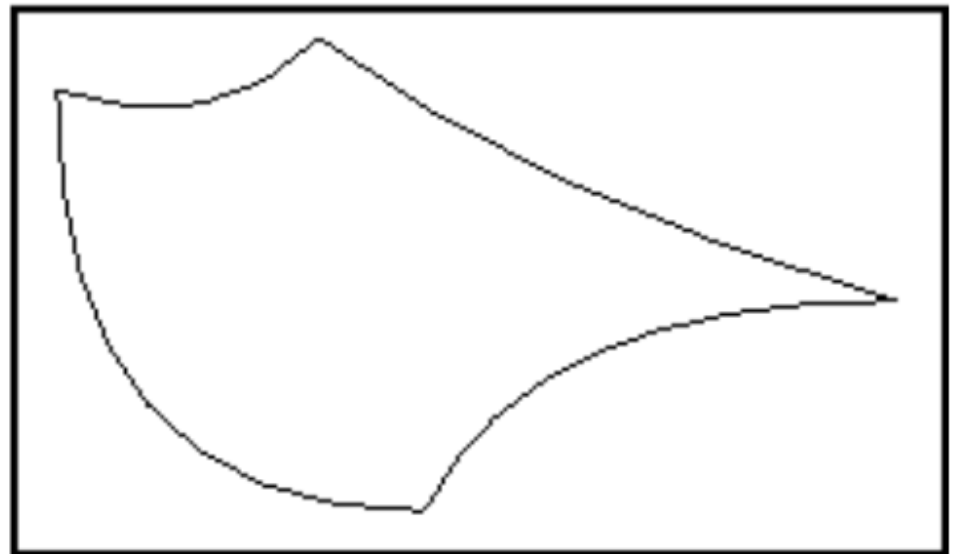
# Convex Hull

- Convex combinations are an extremely important concept in computer graphics and geometric modeling
- The convex-hull concept will allow us to take a set of points and put a **bounding box** about the set of points.
  - Since the bounding box is convex, we are insured that the convex-hull of the set of points is also contained in the bounding box
- Bounding boxes allow us to “keep track of” objects without having to continually reference the object's complex mathematical definition
  - In many cases, a bounding box can be placed about the object and the algorithms can refer to the box when necessary, rather than the object.

# Bounding Box

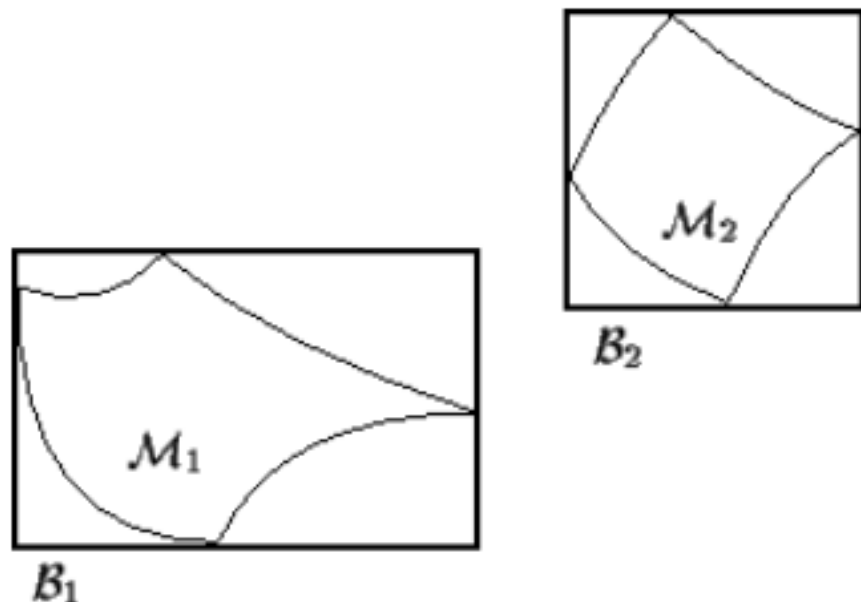
- A bounding box for an object is just a rectangular box in three-dimensional space, with sides parallel to the coordinate planes, that contains (or surrounds) the object. This illustration below shows a two-dimensional box surrounding a curved object.

*BoundingBox*



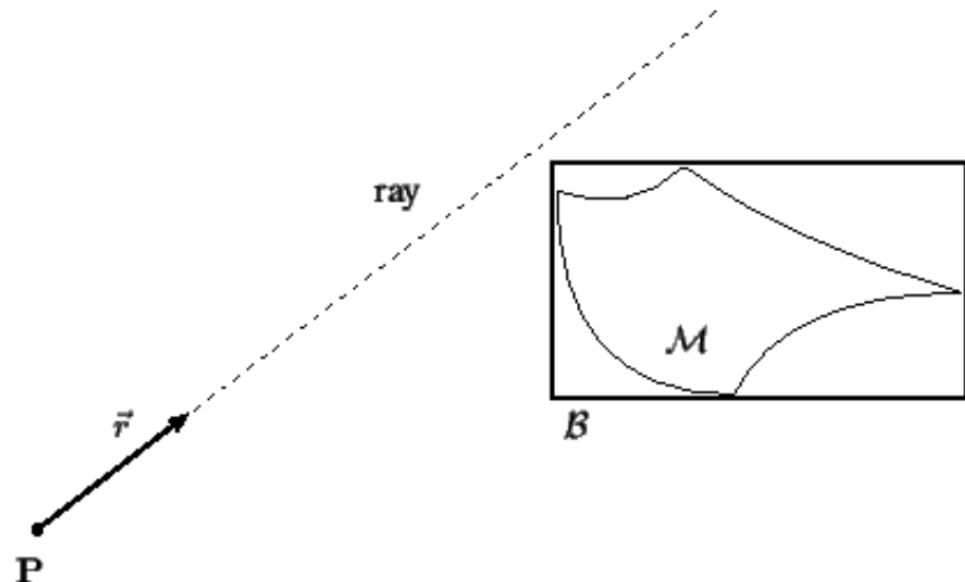
# A Simple Intersection Test

- If we have two complex models  $M_1$  and  $M_2$  and we wish to see if these models do not intersect, we can use a “bounding-box test” to give a quick initial answer.
- If  $B_1$  and  $B_2$  are bounding boxes containing  $M_1$  and  $M_2$  respectively, it is easily seen that  $M_1$  and  $M_2$  cannot intersect if the two bounding boxes do not intersect.



# A Ray/Object Intersection Test

- Want to see if a ray intersects a model  $M$ . This is normally a complex operation, and we can simplify it somewhat by using a simple “bounding-box test” to see if the ray misses  $M$ .
- By placing a bounding box  $B$  around  $M$ , we first see if the ray hits  $B$ , and if not, we know that the ray does not hit the model  $M$ .
- Of course, if the ray hits the bounding box, we then must test it against  $M$  for intersection which may be expensive. But by testing first against the bounding box, we can eliminate a number of complex expensive calculations.

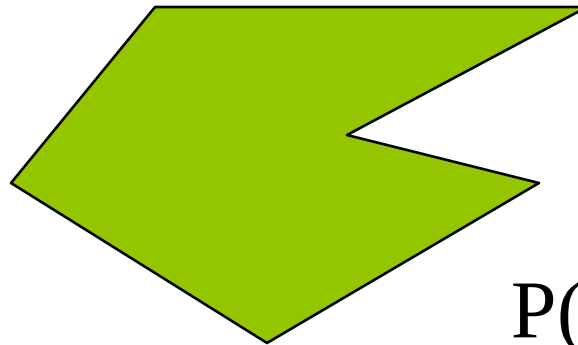


# Curves and Surfaces

- Curves are one parameter entities of the form  $P(\alpha)$  where the function is nonlinear
  - What if the function is linear?
- Surfaces are formed from two-parameter functions  $P(\alpha, \beta)$ 
  - Linear functions give flat surfaces: planes and polygons



$P(\alpha)$

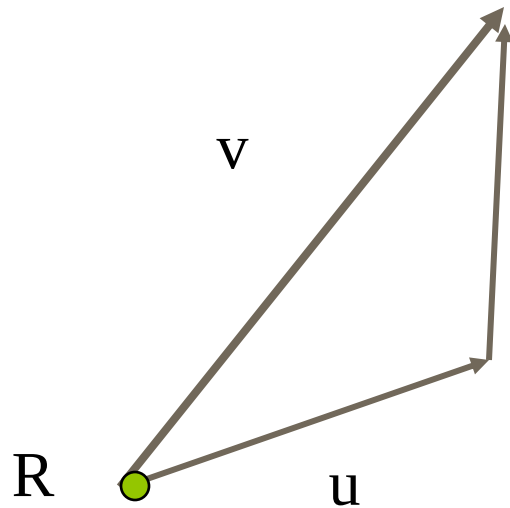


$P(\alpha, \beta)$

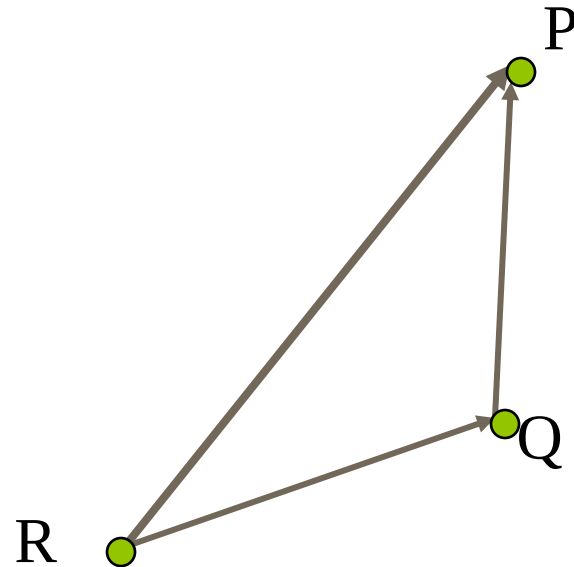
# Planes

A plane can be defined

- a) by a point and two vectors
- b) by three points

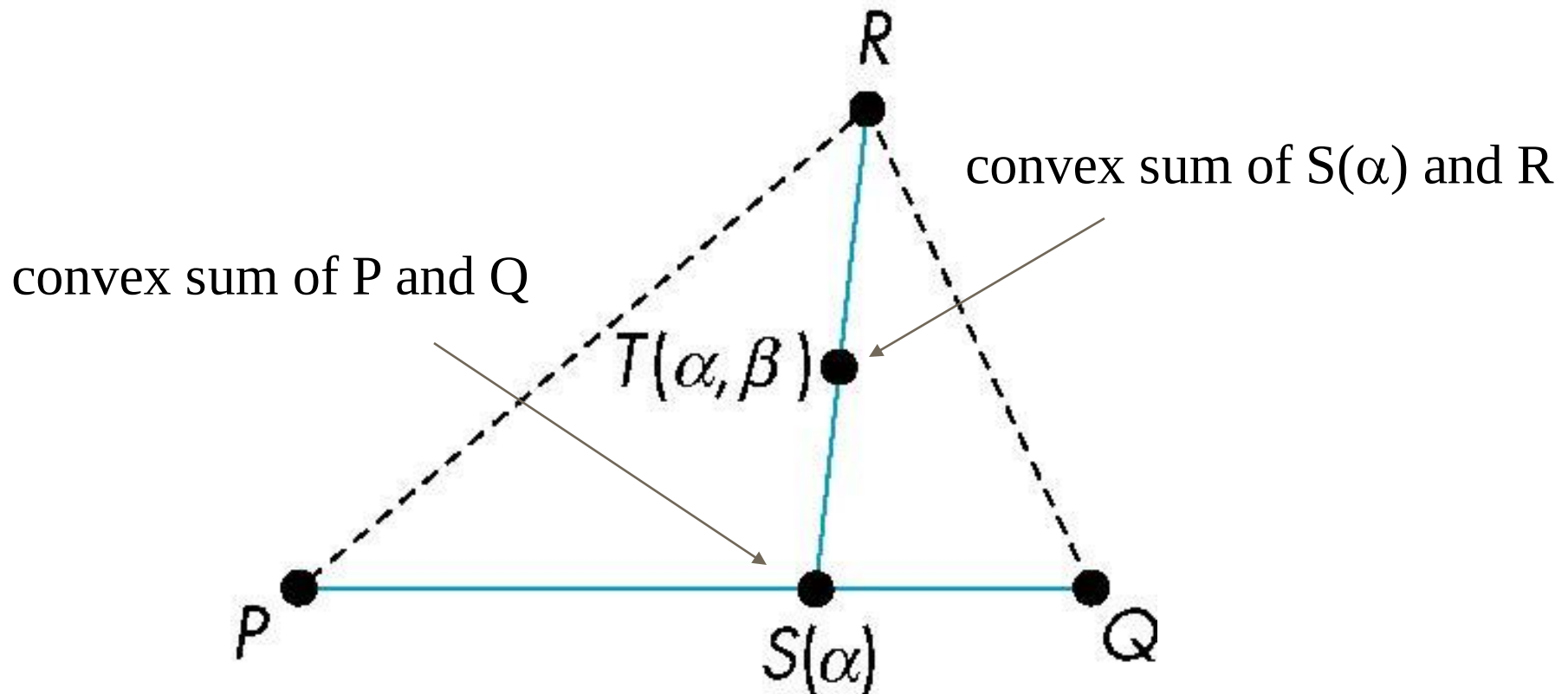


$$P(\alpha, \beta) = R + \alpha u + \beta v$$



$$P(\alpha, \beta) = R + \alpha(Q - R) + \beta(P - R)$$

# Triangles



for  $0 \leq \{\alpha, \beta\} \leq 1$ , we get all points in triangle

# Barycentric Coordinates

Triangle is convex so any point inside can be represented as an affine combination

$$P(\alpha_1, \alpha_2, \alpha_3) = \alpha_1 P + \alpha_2 Q + \alpha_3 R$$

where

$$\alpha_1 + \alpha_2 + \alpha_3 = 1$$

$$\alpha_i \geq 0$$

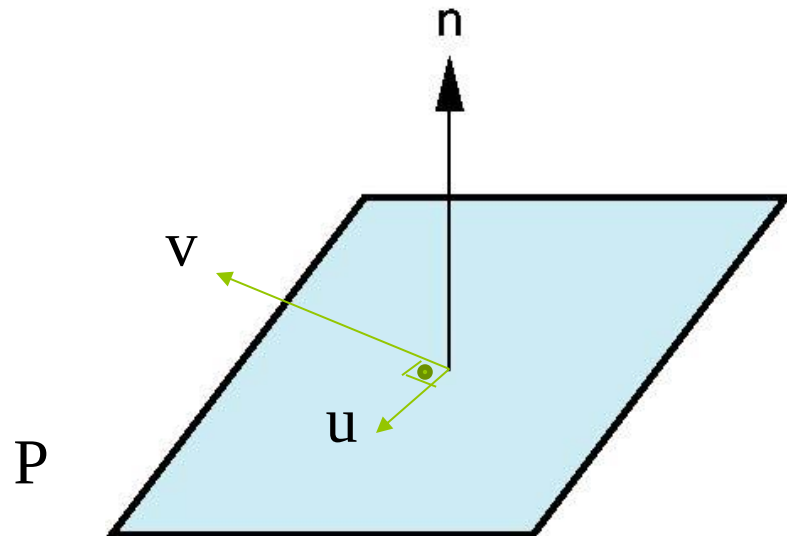
The representation is called the **barycentric coordinate** representation of P

>>  $\alpha_1, \alpha_2, \alpha_3$  are the *barycentric coordinates* of the triangle

# Normals

- Every plane has a vector  $\mathbf{n}$  normal (perpendicular, orthogonal) to it
- From point & two vector form:  $P(\alpha, \beta) = R + \alpha\mathbf{u} + \beta\mathbf{v}$ , we know we can use the cross product to find  $\mathbf{n} = \mathbf{u} \times \mathbf{v}$  and the equivalent form

$$(P(\alpha) - P) \cdot \mathbf{n} = 0$$



# REPRESENTATION

---

**Lecturer: Dr. Ufuk Celikcan**

Based on the slides by: E. Angel and D. Shreiner

# Linear Independence

- A set of vectors  $v_1, v_2, \dots, v_n$  is *linearly independent* when
$$\alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n = 0 \text{ iff } \alpha_1 = \alpha_2 = \dots = 0$$
- If a set of vectors is linearly **independent**, we cannot represent one in terms of the others
- If a set of vectors is linearly **dependent**, at least one of them can be written in terms of the others

# Dimension

- In a vector space, the maximum number of linearly independent vectors is fixed and is called the ***dimension*** of the space
- In an  $n$ -dimensional space, any set of  $n$  linearly independent vectors form a ***basis*** for the space
- Given a basis composed of  $v_1, v_2, \dots, v_n$ , any vector  $v$  can be written as

$$v = \alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n$$

where the set of  $\{\alpha_i\}$  is unique

# Representation

- So far, we have been able to work with geometric entities without using any frame of reference, such as a coordinate system
- Now, we need a frame of reference to relate points and objects to the physical world.
  - For example, where is a point exactly?
  - Can't answer without a reference system such as
    - World coordinates
    - Camera coordinates

# Coordinate Systems

- Consider a basis  $v_1, v_2, \dots, v_n$  in a vector space
- A vector in that space can be decomposed as

$$v = \alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n$$

- Here, the list of scalars  $\{\alpha_1, \alpha_2, \dots, \alpha_n\}$  is the ***representation*** of  $v$  with respect to the given basis
- We can write the representation as a row or a column array of scalars

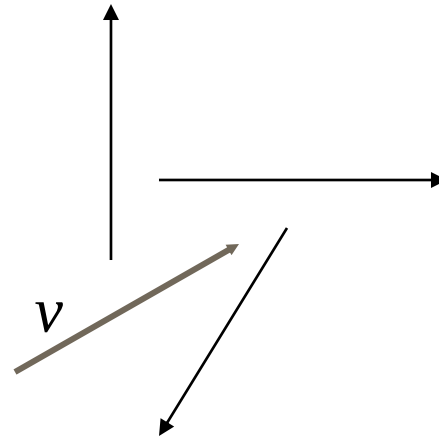
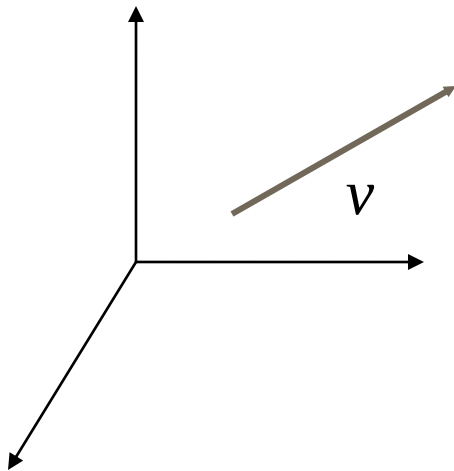
$$\mathbf{a} = [\alpha_1 \ \alpha_2 \ \dots \ \alpha_n]^T = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{bmatrix}$$

# Example

- $\mathbf{v} = 2\mathbf{v}_1 + 3\mathbf{v}_2 - 4\mathbf{v}_3$
- $\mathbf{a} = ?$
- $\mathbf{a} = [2 \ 3 \ -4]^T$
- Note that this representation is with respect to a particular basis.

# Coordinate Systems

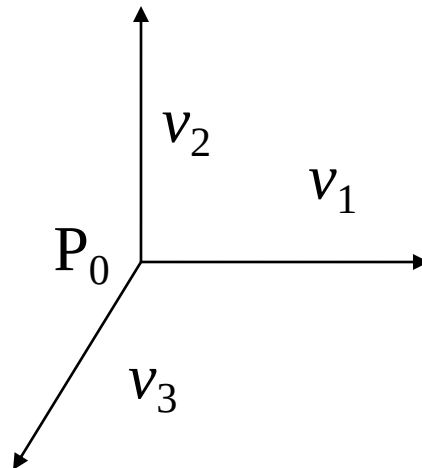
- Which is correct?



- Both are correct, because vectors have no fixed location

# Frames

- A coordinate system by itself is insufficient to represent points
- If we work in an affine space, we can add a single point, the ***origin***, to the **basis vectors** to form a **frame**
- **frame** : ***basis + origin***



# Representation in a 3D Frame

- Frame determined by  $(P_0, v_1, v_2, v_3)$
- Within this frame, every vector can be written as

$$v = \alpha_1 v_1 + \alpha_2 v_2 + \alpha_3 v_3$$

- And every point can be written as

$$P = P_0 + \beta_1 v_1 + \beta_2 v_2 + \beta_3 v_3$$

# Confusing Points and Vectors

Consider the point  $P$  and the vector  $\mathbf{v}$

$$P = P_0 + \beta_1 v_1 + \beta_2 v_2 + \dots + \beta_n v_n$$

$$\mathbf{v} = \alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n$$

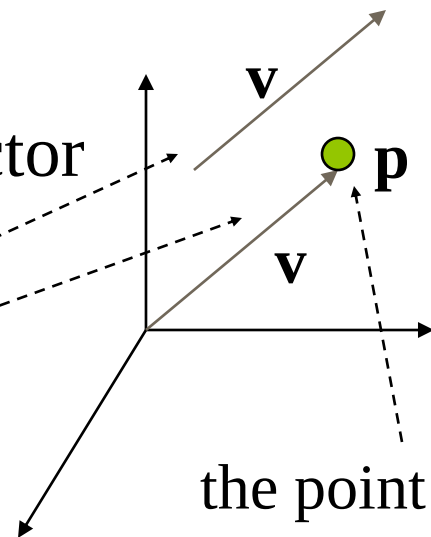
They appear to have similar representations

$$\mathbf{P} = [\beta_1 \ \beta_2 \ \beta_3] \quad \mathbf{v} = [\alpha_1 \ \alpha_2 \ \alpha_3]$$

which confuses the point with the vector

**A vector has no position**

the same vector defined above  
can be placed anywhere



the point is fixed

# One Representation to Rule Them All!

>> Using the features of affine spaces:

$$0 \bullet P = \mathbf{0} \text{ and } 1 \bullet P = P$$

we can write

$$\mathbf{v} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \alpha_3 \mathbf{v}_3 = [\alpha_1 \ \alpha_2 \ \alpha_3 \ 0] [\mathbf{v}_1 \ \mathbf{v}_2 \ \mathbf{v}_3 \ P_0]^T$$

$$P = P_0 + \beta_1 \mathbf{v}_1 + \beta_2 \mathbf{v}_2 + \beta_3 \mathbf{v}_3 = [\beta_1 \ \beta_2 \ \beta_3 \ 1] [\mathbf{v}_1 \ \mathbf{v}_2 \ \mathbf{v}_3 \ P_0]^T$$

Thus we obtain the four-dimensional  
***homogeneous coordinate*** representation

$$\mathbf{v} = [\alpha_1 \ \alpha_2 \ \alpha_3 \ 0]^T$$

$$\mathbf{p} = [\beta_1 \ \beta_2 \ \beta_3 \ 1]^T$$

# Homogeneous Coordinates

- The homogeneous coordinate form of a three dimensional point  $[x \ y \ z]$  is given as

$$\mathbf{p} = [x' \ y' \ z' \ w]^T = [wx \ wy \ wz \ w]^T$$

- We return to a three dimensional representation (for  $w \neq 0$ ) by

$$x \leftarrow x'/w$$

$$y \leftarrow y'/w$$

$$z \leftarrow z'/w$$

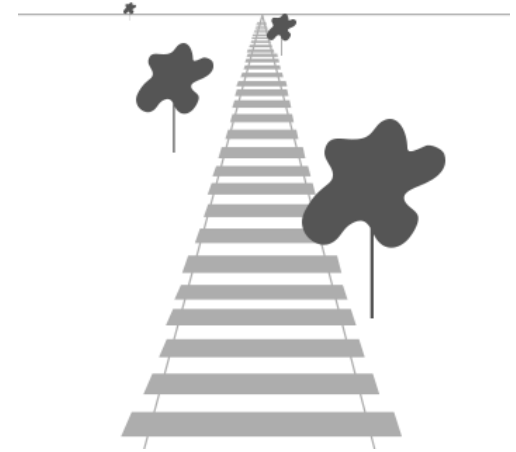
- If  $w=0$ , the representation is of a vector
- For  $w=1$ , the representation of a point is  $[x \ y \ z \ 1]$

# Homogeneous Coordinates and Computer Graphics

- Homogeneous coordinates are key to all computer graphics systems
- All standard transformations (rotation, translation, scaling) can be implemented with matrix multiplications using  $4 \times 4$  matrices
- **Hardware pipeline works with 4 dimensional representations**
- For orthographic viewing, we can maintain  $w=0$  for vectors and  $w=1$  for points
- For perspective viewing, we apply a *perspective division*

# Perspective Division

- Perspective projection tries to mimic this effect using a perspective projection matrix.
- The projection matrix maps a given frustum range to clip space
- also manipulates the  $w$  value of each vertex coordinate so that the further away a vertex coordinate is from the viewer, the higher this  $w$  component becomes

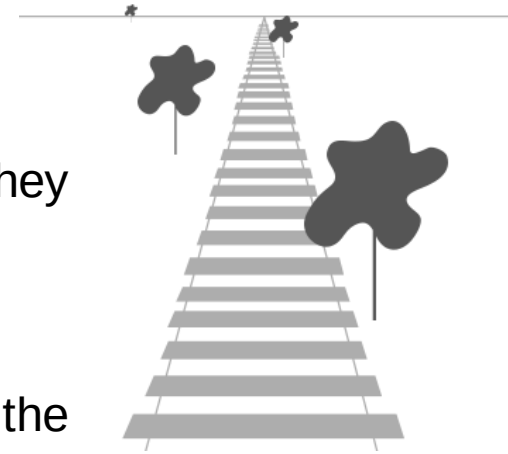


# Perspective Division

- When the coordinates are transformed to the clip space, they are in the range between -w and w.
  - Anything outside this range is clipped out.
- WebGL requires that the visible coordinates fall between the range -1 and +1 (normalized device coordinates) as the final vertex shader output
- So, once the coordinates are in clip space, perspective division is applied to the clip space coordinates:

$$out = \begin{pmatrix} x/w \\ y/w \\ z/w \end{pmatrix}$$

- Each component of the vertex coordinate is divided by its w component giving smaller vertex coordinates the further away a vertex is from the viewer.
- The resulting coordinates are then in normalized device space



# Change of Coordinate Systems

Consider two representations of the same vector  $\mathbf{v}$  with respect to two different bases:

$$\mathbf{a} = [\alpha_1 \ \alpha_2 \ \alpha_3]$$

$$\mathbf{b} = [\beta_1 \ \beta_2 \ \beta_3]$$

where

$$\mathbf{v} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \alpha_3 \mathbf{v}_3 = [\alpha_1 \ \alpha_2 \ \alpha_3] [\mathbf{v}_1 \ \mathbf{v}_2 \ \mathbf{v}_3]^T$$

$$= \beta_1 \mathbf{u}_1 + \beta_2 \mathbf{u}_2 + \beta_3 \mathbf{u}_3 = [\beta_1 \ \beta_2 \ \beta_3] [\mathbf{u}_1 \ \mathbf{u}_2 \ \mathbf{u}_3]^T$$

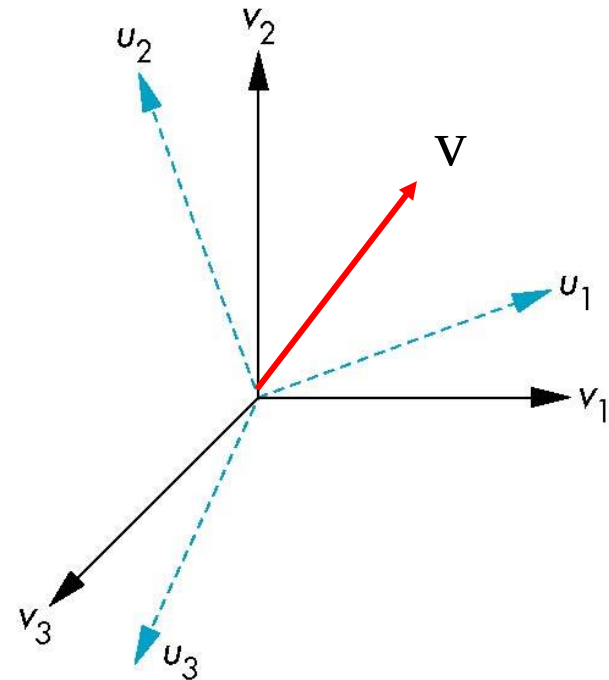
# Representing one basis in terms of another

Each of the basis vectors  $u_1, u_2, u_3$  is a vector that can be represented in terms of the first basis  $v_1, v_2, v_3$

$$u_1 = \gamma_{11}v_1 + \gamma_{12}v_2 + \gamma_{13}v_3$$

$$u_2 = \gamma_{21}v_1 + \gamma_{22}v_2 + \gamma_{23}v_3$$

$$u_3 = \gamma_{31}v_1 + \gamma_{32}v_2 + \gamma_{33}v_3$$



# Matrix Form

These coefficients define a 3 x 3 matrix

$$\mathbf{M} = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} \\ \gamma_{21} & \gamma_{22} & \gamma_{23} \\ \gamma_{31} & \gamma_{32} & \gamma_{33} \end{bmatrix}$$

thus the bases  $\mathbf{u}$  and  $\mathbf{v}$  can be related by

$$\mathbf{u} = \mathbf{M}^T \mathbf{v}$$

(see the textbook for numerical examples)

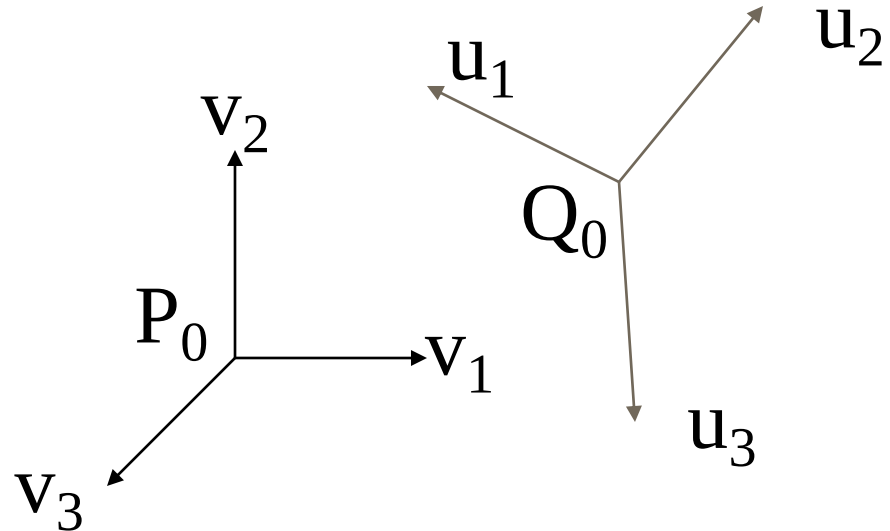
# Change of Frames

- We can apply a similar process in homogeneous coordinates to the representations of both points and vectors

Consider two frames:

$(P_0, v_1, v_2, v_3)$

$(Q_0, u_1, u_2, u_3)$



- Any point or vector can be represented in either frame
- We can represent  $Q_0, u_1, u_2, u_3$  in terms of  $P_0, v_1, v_2, v_3$

# Representing One Frame in Terms of the Other

Extending what we did with change of bases

$$u_1 = \gamma_{11}v_1 + \gamma_{12}v_2 + \gamma_{13}v_3$$

$$u_2 = \gamma_{21}v_1 + \gamma_{22}v_2 + \gamma_{23}v_3$$

$$u_3 = \gamma_{31}v_1 + \gamma_{32}v_2 + \gamma_{33}v_3$$

$$Q_0 = \gamma_{41}v_1 + \gamma_{42}v_2 + \gamma_{43}v_3 + \gamma_{44}P_0$$

defining a 4 x 4 matrix

$$\mathbf{M} = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} & 0 \\ \gamma_{21} & \gamma_{22} & \gamma_{23} & 0 \\ \gamma_{31} & \gamma_{32} & \gamma_{33} & 0 \\ \gamma_{41} & \gamma_{42} & \gamma_{43} & \gamma_{44} \end{bmatrix}$$

# Working with Representations

Within the two frames, any point or vector has a representation of the same form

$\mathbf{a} = [\alpha_1 \ \alpha_2 \ \alpha_3 \ \alpha_4]^T$  in one frame

$\mathbf{b} = [\beta_1 \ \beta_2 \ \beta_3 \ \beta_4]^T$  in the other frame

where  $\alpha_4 = \beta_4 = 1$  for points and  $\alpha_4 = \beta_4 = 0$  for vectors and

$$\mathbf{a} = \mathbf{M}^T \mathbf{b}$$

The matrix  $\mathbf{M}$  is 4 x 4 and specifies an affine transformation in homogeneous coordinates

# TRANSFORMATIONS

---

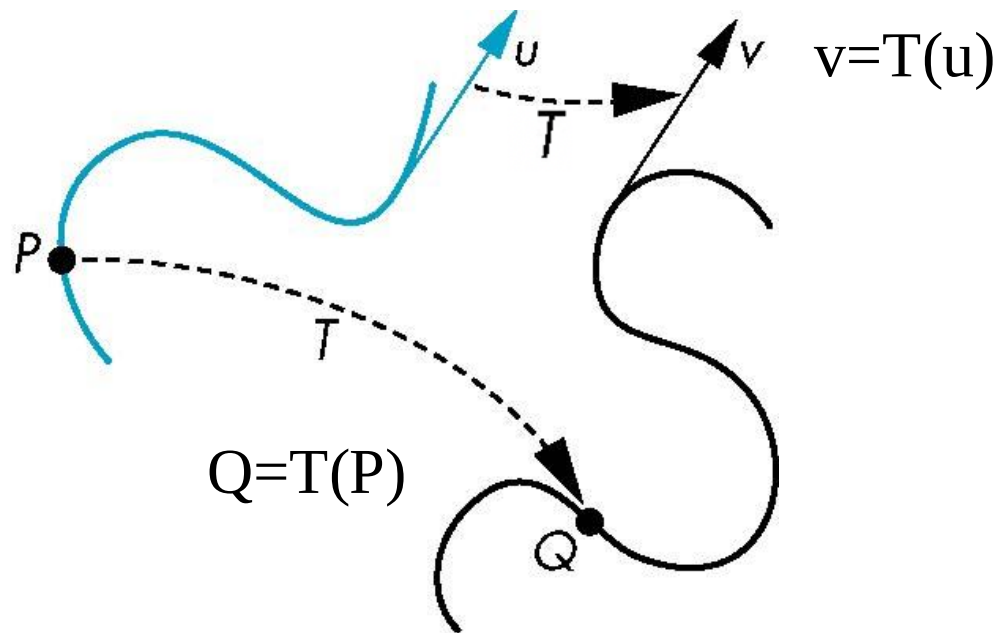
**Lecturer: Dr. Ufuk Celikcan**

Based on the slides by: E. Angel and D. Shreiner

# General Transformations

## A transformation

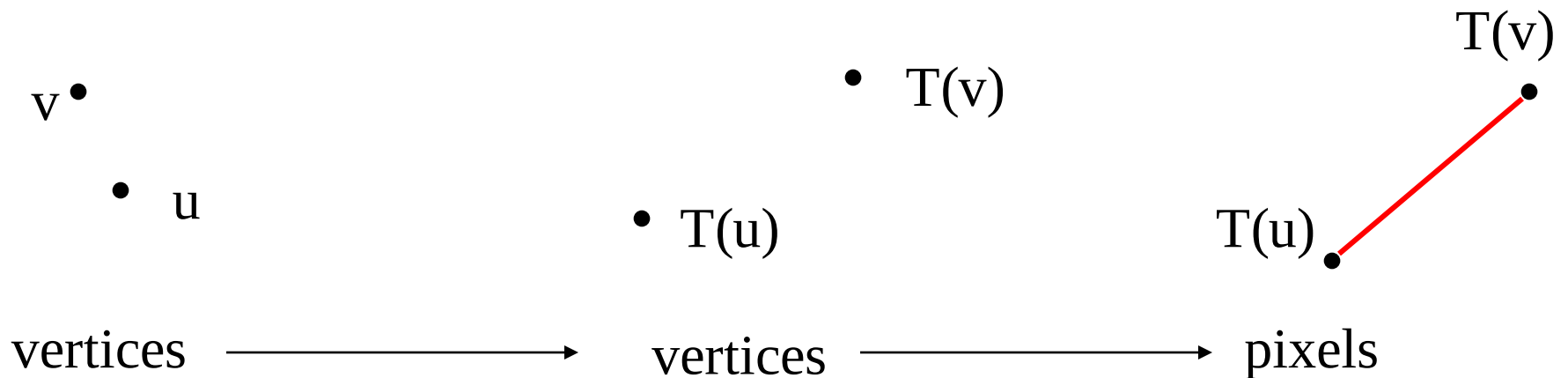
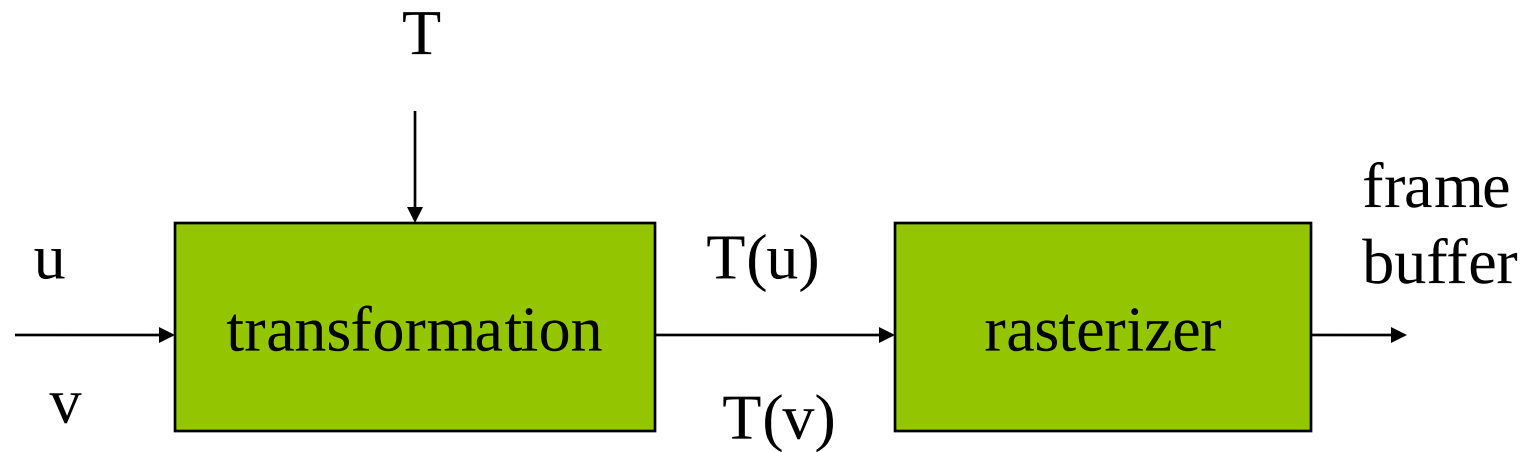
- maps points to other points
- and/or maps vectors to other vectors



# Affine Transformations

- Line preserving
- Characteristic of many physically important transformations
  - **Rigid body transformations:** rotation, translation
  - Scaling, shear
- **Importance in computer graphics is that:**
  - we need to transform only endpoints of line segments
  - and let implementation draw line segment between the transformed endpoints

# Pipeline Implementation



# Linear Transformations vs. Affine Transformations

- Every linear transformation is equivalent to a change in frames
- Every linear transformation is affine, but not every affine transformation is linear.
- **Every affine transformation preserves lines and parallelism** (but not necessarily distances and angles).
  - >> **preserves collinearity**
  - >> Affine Transformations
    - transform **parallel lines into parallel lines**
    - and **preserve ratios of distances along parallel lines**
- However, an affine transformation has **only 12 degrees of freedom** because 4 of the elements in the matrix are fixed.
- Hence affine transformations are a subset of all possible 4x4 linear transformations

# Notation

We will be working with both: coordinate-free representations of transformations and representations within a particular frame

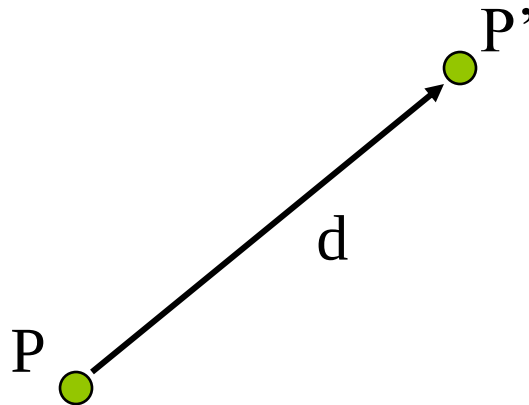
Our choice of notation:

- $P, Q, R$ : points in an affine space
- $u, v, w$ : vectors in an affine space
- $\alpha, \beta, \gamma$ : scalars
- **$p, q, r$** : representations of points  
-array of 4 scalars in homogeneous coordinates
- **$u, v, w$** : representations of vectors  
-array of 4 scalars in homogeneous coordinates

sometimes I  
forget to make the  
representations  
bold 🧐♂ so  
don't get too hung  
up on this 🧐♂

# Translation

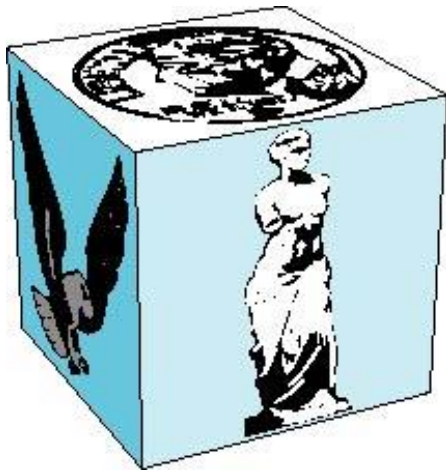
- Move (translate, displace) a point to a new location



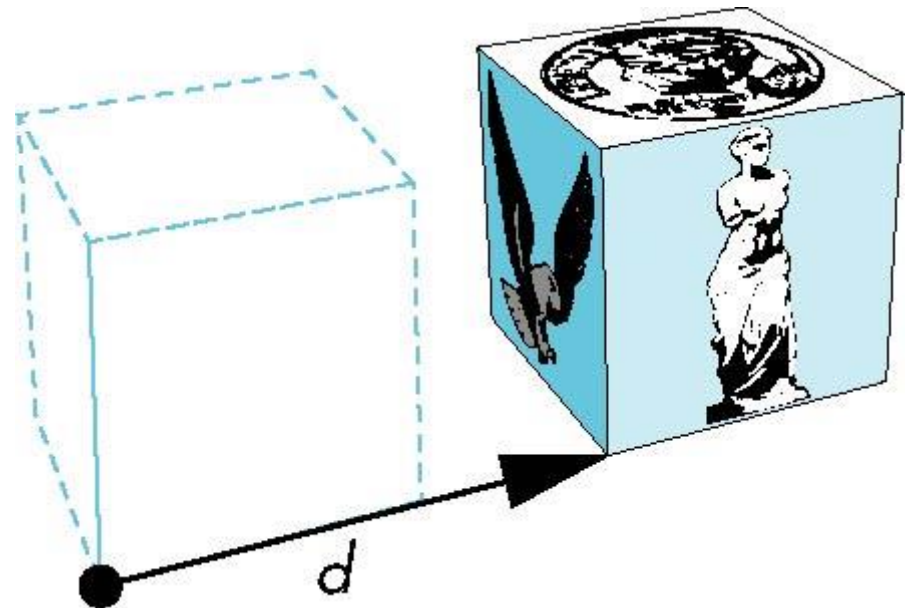
- Displacement determined by a vector  $d$ 
  - Three degrees of freedom
  - $P' = P + d$

# How many ways?

Although we can move a single point to a new location in infinite ways, when we move many points there is usually only one way



object



translation: every point displaced  
by same vector

# Translation Using Representations

Using the homogeneous coordinate representation in some frame

$$\mathbf{p} = [x \ y \ z \ 1]^T$$

$$\mathbf{p}' = [x' \ y' \ z' \ 1]^T$$

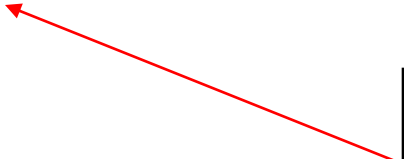
$$\mathbf{d} = [dx \ dy \ dz \ 0]^T$$

Hence  $\mathbf{p}' = \mathbf{p} + \mathbf{d}$  or

$$x' = x + d_x$$

$$y' = y + d_y$$

$$z' = z + d_z$$



note that this expression is in four dimensions and expresses  $\text{point}_2 = \text{point}_1 + \text{vector}$

# Translation Matrix

We can express translation using a 4 x 4 matrix  $\mathbf{T}$  in homogeneous coordinates  $\mathbf{p}' = \mathbf{T}\mathbf{p}$  where

$$\mathbf{T} = \mathbf{T}(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

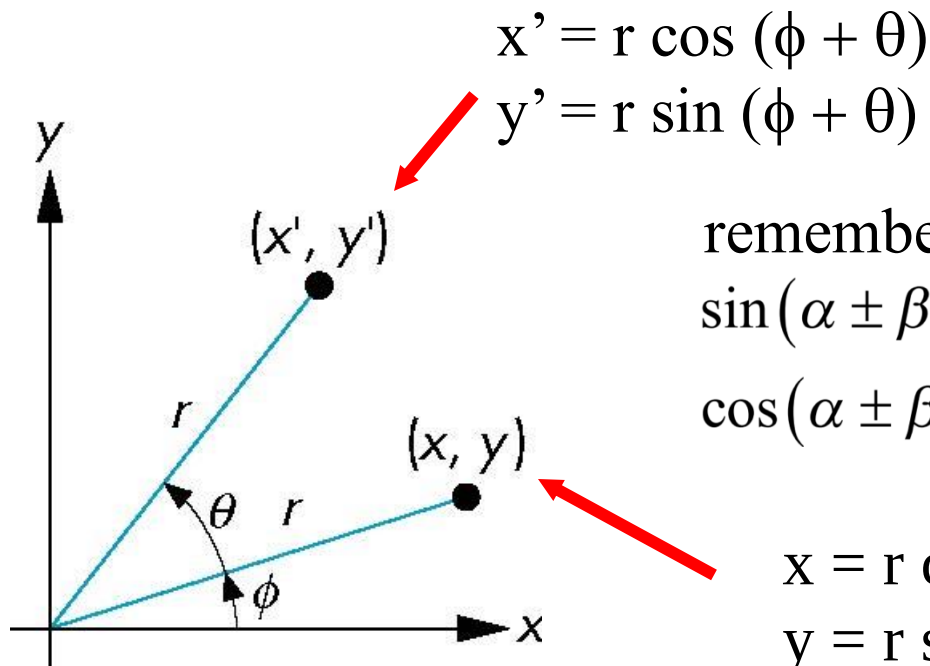
This form is better for implementation because

- all affine transformations can be expressed this way
- and multiple transformations can be concatenated together

# Rotation (2D)

Consider rotation about the origin on the  $xy$  plane by  $\theta$  degrees

- radius stays the same, angle increases by  $\theta$



$$x' = r \cos (\phi + \theta)$$

$$y' = r \sin (\phi + \theta)$$

remember

$$\sin (\alpha \pm \beta) = \sin \alpha \cos \beta \pm \cos \alpha \sin \beta$$

$$\cos (\alpha \pm \beta) = \cos \alpha \cos \beta \mp \sin \alpha \sin \beta$$

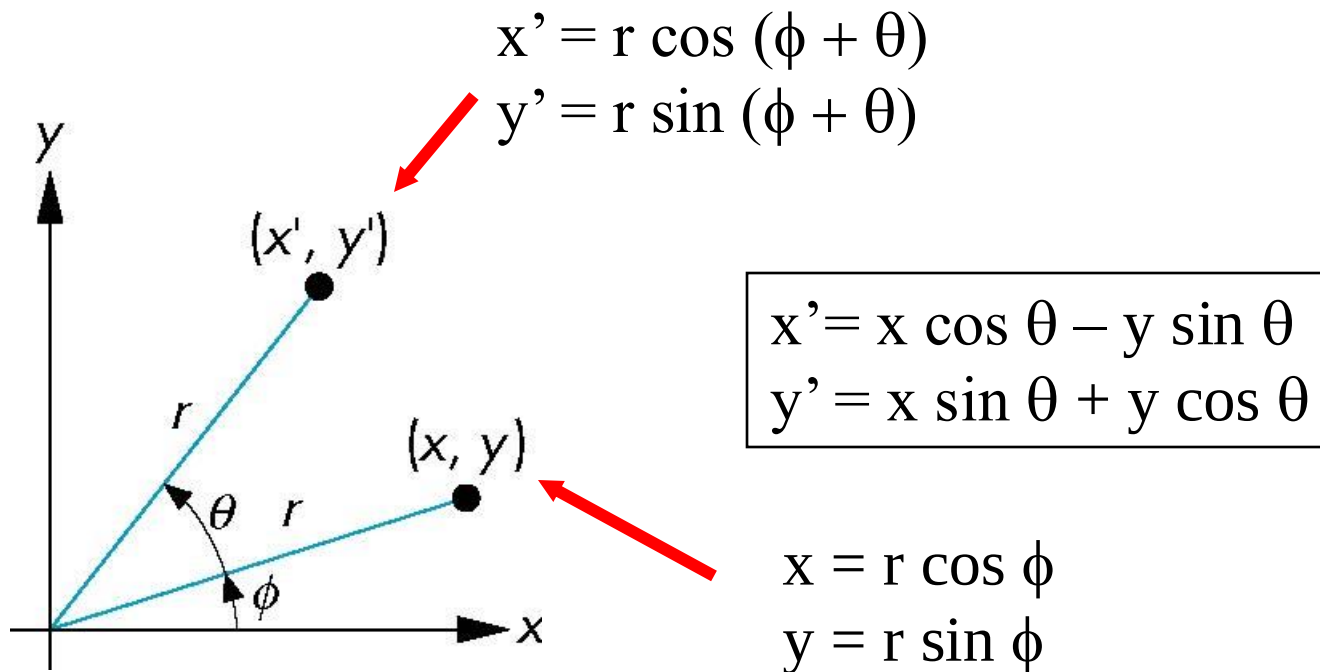
$$x = r \cos \phi$$

$$y = r \sin \phi$$

# Rotation (2D)

Consider rotation about the origin on the  $xy$  plane by  $\theta$  degrees

- radius stays the same, angle increases by  $\theta$



# Rotation about the z-axis

- Rotation on the xy plane in three dimensions =  
Rotation about the z-axis:
  - Leaves all points with the same  $z$
  - Equivalent to rotation in two dimensions in planes of constant  $z$

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

$$z' = z$$

- or in homogeneous coordinates

$$\mathbf{p}' = \mathbf{R}_z(\theta) \mathbf{p}$$

# Rotation Matrix

$$\mathbf{R} = \mathbf{R}_Z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

# Rotation about x and y axes

- Same argument as for rotation about z-axis
  - For rotation about x-axis >> x is unchanged
  - For rotation about y-axis >> y is unchanged

$$\mathbf{R} = \mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{R} = \mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

# Scaling

Expand or contract along each axis (fixed point of origin)

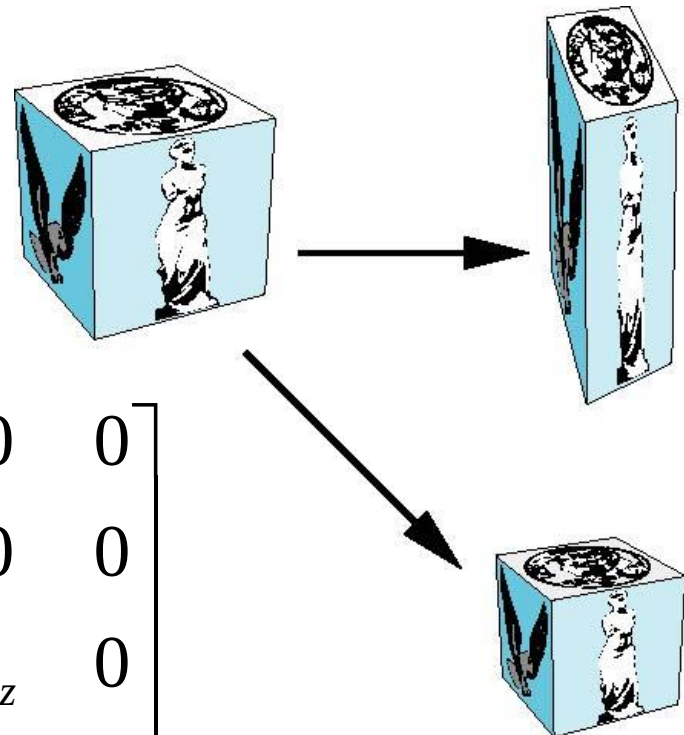
$$x' = s_x x$$

$$y' = s_y y$$

$$z' = s_z z$$

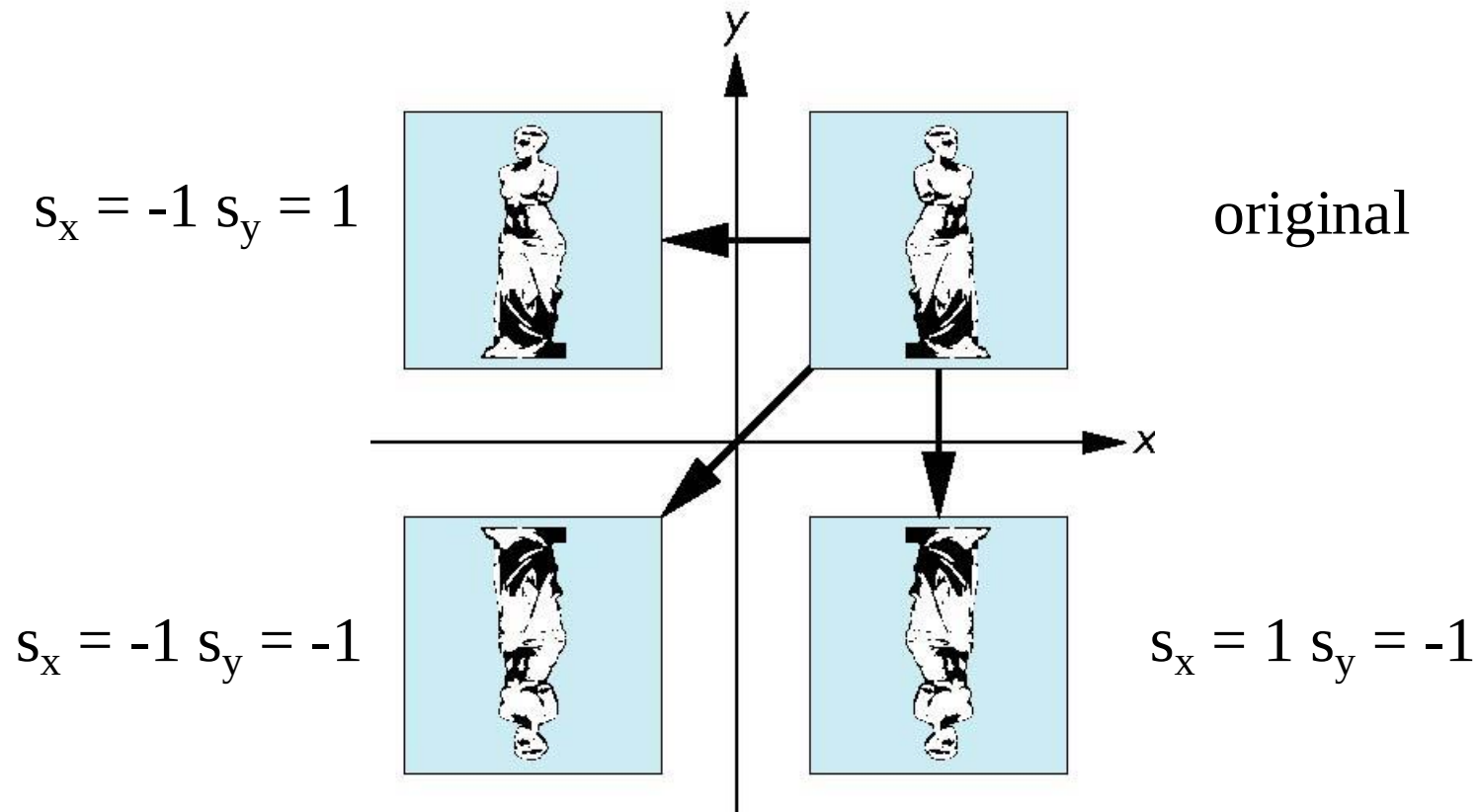
$$\mathbf{p}' = \mathbf{S}\mathbf{p}$$

$$\mathbf{S} = \mathbf{S}(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



# Reflection

corresponds to negative scale factors



# Inverses

- Although we could compute inverse matrices using linear algebra, we can instead use simple geometric observations as follows:
  - Translation:  $\mathbf{T}^{-1}(d_x, d_y, d_z) = \mathbf{T}(-d_x, -d_y, -d_z)$
  - Rotation:  $\mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta)$ 
    - Holds for any rotation matrix
    - Note that since  $\cos(-\theta) = \cos(\theta)$  and  $\sin(-\theta) = -\sin(\theta)$   
>>  $\mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta) = \mathbf{R}^T(\theta)$  : transpose of  $\mathbf{R}(\theta)$
  - Scaling:  $\mathbf{S}^{-1}(s_x, s_y, s_z) = \mathbf{S}(1/s_x, 1/s_y, 1/s_z)$

# Concatenation

- We can form arbitrary affine transformation matrices by multiplying together rotation, translation, and scaling matrices in any order and more than once
- Because the same transformation is applied to many vertices, the cost of forming a matrix  $\mathbf{M}=\mathbf{ABCD}$  once is insignificant compared to the cost of computing  $\mathbf{Mp}$  for many vertices  $\mathbf{p}$
- The challenging part is how to form a desired transformation from the specifications in the application

# Order of Transformations

- Note that the rightmost matrix is the one first applied
- Mathematically, the following are equivalent

$$\mathbf{p}' = \mathbf{ABCp} = \mathbf{A(B(Cp))}$$

- Note that many references use column matrices to represent points. In terms of column matrices

$$\mathbf{p}'^T = \mathbf{p}^T \mathbf{C}^T \mathbf{B}^T \mathbf{A}^T$$

# General Rotation About the Origin

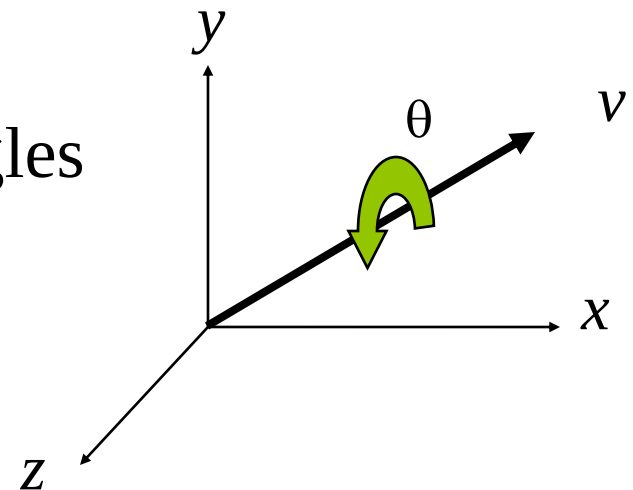
A rotation by  $\theta$  about an arbitrary axis can be decomposed into the concatenation of rotations about the  $x$ ,  $y$ , and  $z$  axes

$$\mathbf{R}(\theta) = \mathbf{R}_z(\theta_z) \mathbf{R}_y(\theta_y) \mathbf{R}_x(\theta_x)$$

$\theta_x$ ,  $\theta_y$  and  $\theta_z$  are called the Euler angles

Note: Rotations do not commute.

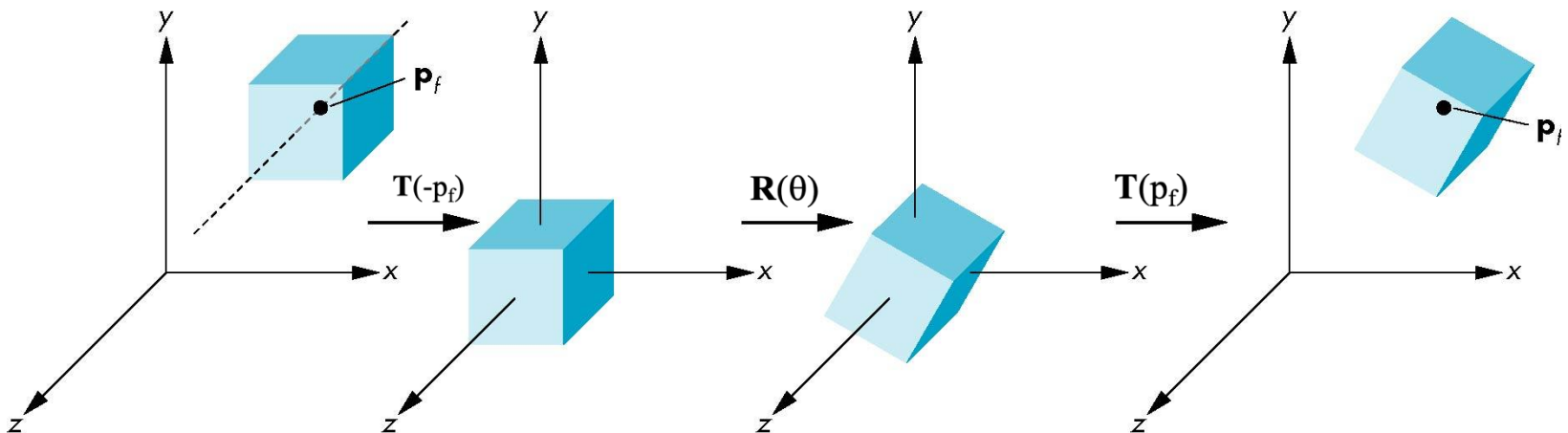
>> We can use rotations in another order but with different angles.



# Rotation About a Point other than the Origin

1. Move from the fixed point to origin
2. Rotate
3. Move back to the fixed point

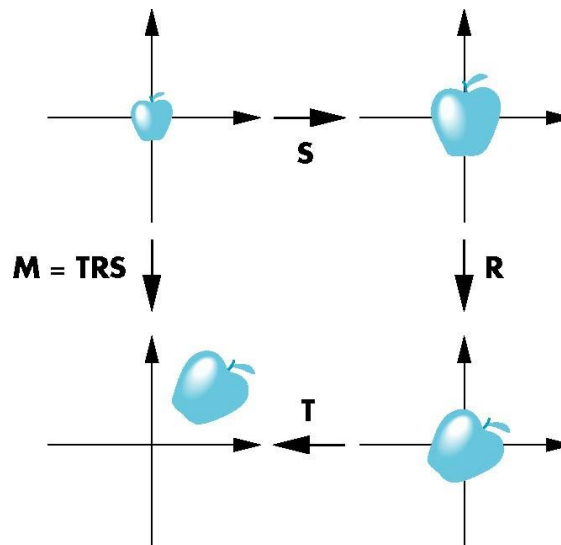
$$\mathbf{M} = \mathbf{T}(\mathbf{p}_f) \mathbf{R}(\theta) \mathbf{T}(-\mathbf{p}_f)$$



# Instancing

- In modeling, we often start with a simple object centered at the origin, oriented with the axes, and at a standard size
- We apply an ***instance transformation*** to its vertices in order to

1. Scale
2. Orient
3. Move



# Instancing

- In modeling, we often start with a simple object centered at the origin, oriented with the axis, and at a standard size
- We apply an **instance transformation** to its vertices in order to

1. Scale (S)
2. Orient (R)
3. Move (T)

$$\begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R_{00} & R_{01} & R_{02} & 0 \\ R_{10} & R_{11} & R_{12} & 0 \\ R_{20} & R_{21} & R_{22} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{\text{instancing}} = \text{TRS} =$$

$$\begin{bmatrix} S_x R_{00} & R_{01} & R_{02} & T_x \\ R_{10} & S_y R_{11} & R_{12} & T_y \\ R_{20} & R_{21} & S_z R_{22} & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

# TR

$$\begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} R_{00} & R_{01} & R_{02} & 0 \\ R_{10} & R_{11} & R_{12} & 0 \\ R_{20} & R_{21} & R_{22} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M = TR = \begin{bmatrix} R_{00} & R_{01} & R_{02} & T_X \\ R_{10} & R_{11} & R_{12} & T_Y \\ R_{20} & R_{21} & R_{22} & T_Z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M = TR = \begin{bmatrix} \mathbf{R}_{3 \times 3} & \mathbf{T}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}$$

# Rotation Matrices

- Rotation matrices have several special properties that we can check to validate whether a given matrix can be a rotation matrix.
- While a matrix still may not be a rotation matrix even if it passes all these checks, it is definitely not a rotation matrix if it fails even one of these:

- The determinant of  $\mathbf{R}$  equals one.

$$\mathbf{A} = \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}$$

- The inverse of  $\mathbf{R}$  is its transpose  $\mathbf{R}^T = \mathbf{R}^{-1}$  so  $\mathbf{R}^T \cdot \mathbf{R} = \mathbf{I}$

$$\det(\mathbf{A}) = a_1 \begin{vmatrix} b_2 & b_3 \\ c_2 & c_3 \end{vmatrix} - a_2 \begin{vmatrix} b_1 & b_3 \\ c_1 & c_3 \end{vmatrix} + a_3 \begin{vmatrix} b_1 & b_2 \\ c_1 & c_2 \end{vmatrix}$$

$$= a_1(b_2c_3 - b_3c_2) - a_2(b_1c_3 - b_3c_1) + a_3(b_1c_2 - b_2c_1)$$

- The dot product of any row or column with itself equals one.

$$\text{Ex: } (\cos \theta \mathbf{i} + \sin \theta \mathbf{j}) \cdot (\cos \theta \mathbf{i} + \sin \theta \mathbf{j}) = 1$$

- The dot product of any row with any other row equals zero.

$$\text{Ex: } (\cos \theta \mathbf{i} + \sin \theta \mathbf{j}) \cdot (-\sin \theta \mathbf{i} + \cos \theta \mathbf{j}) = 0$$

- The dot product of any column with any other column equals zero.

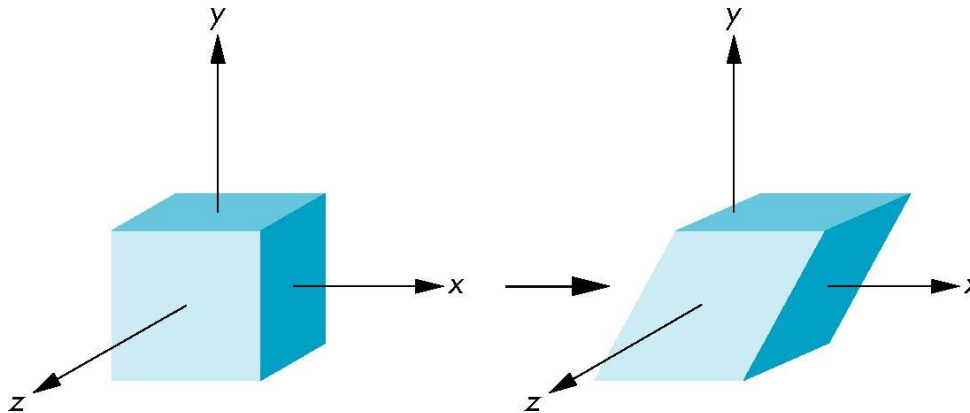
$$\text{Ex: } (\cos \theta \mathbf{i} - \sin \theta \mathbf{j}) \cdot (\sin \theta \mathbf{i} + \cos \theta \mathbf{j}) = 0$$

For example,  $\mathbf{R}$  given below satisfies all

$$\mathbf{R}_{3 \times 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{\sqrt{3}}{2} & \frac{1}{2} \\ 0 & -\frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos 30^\circ & -\sin 30^\circ \\ 0 & \sin 30^\circ & \cos 30^\circ \end{bmatrix}$$

# Shear

- Helpful to add one more basic transformation
- Equivalent to pulling faces in opposite directions



# Shear Matrix

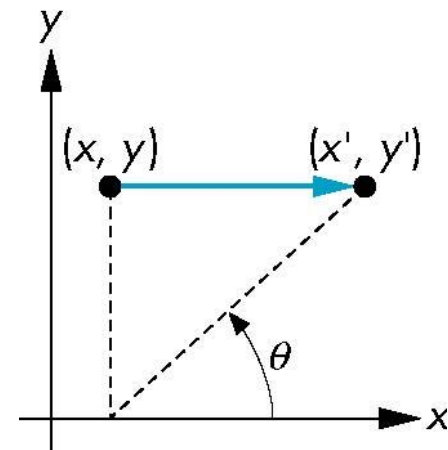
Consider a simple shear along  $x$ -axis

$$x' = x + y \cot \theta$$

$$y' = y$$

$$z' = z$$

$$\mathbf{H}(\theta) = \begin{bmatrix} 1 & \cot \theta & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



# WEBGL TRANSFORMATIONS

---

**Lecturer: Dr. Ufuk Çelikcan**

Based on the slides by: E. Angel and D. Shreiner

# Objectives

- Learn how to carry out transformations in WebGL
  - Rotation
  - Translation
  - Scaling
- Introduce transformations in javascript
  - Model-view
  - Projection

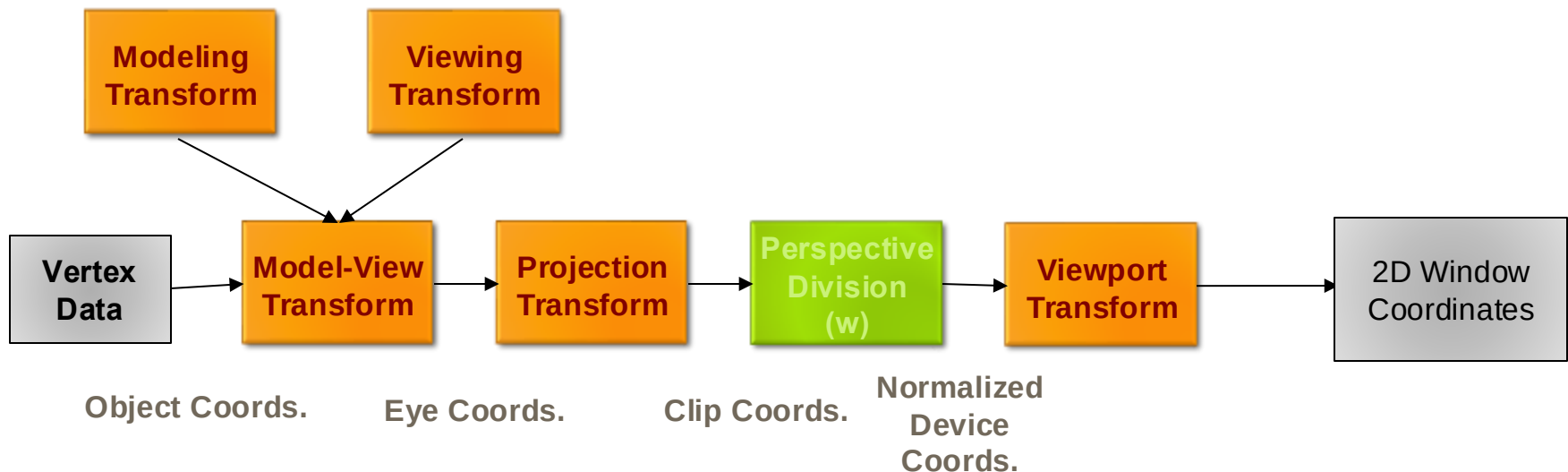
# 3D Transformations

- Vertices are transformed by 4×4 matrices
  - all affine operations are matrix multiplications
- Perspective projections and translations require the 4<sup>th</sup> row and column.
  - Otherwise, these operations would require a vector-addition operation, in addition to the matrix multiplication.
  - **For operations other than perspective projection, the fourth row is always (0, 0, 0, 1) which leaves the w-coordinate unchanged..**
- All matrices are stored column-major in WebGL
  - this is opposite of what “C” programmers expect
- **Matrices are always post-multiplied (i.e., left-multiplied)**
  - product of matrix and vector is  $\mathbf{M}\vec{v}$

$$\mathbf{M} = \begin{bmatrix} m_0 & m_4 & m_8 & m_{12} \\ m_1 & m_5 & m_9 & m_{13} \\ m_2 & m_6 & m_{10} & m_{14} \\ m_3 & m_7 & m_{11} & m_{15} \end{bmatrix}$$

# Transformations

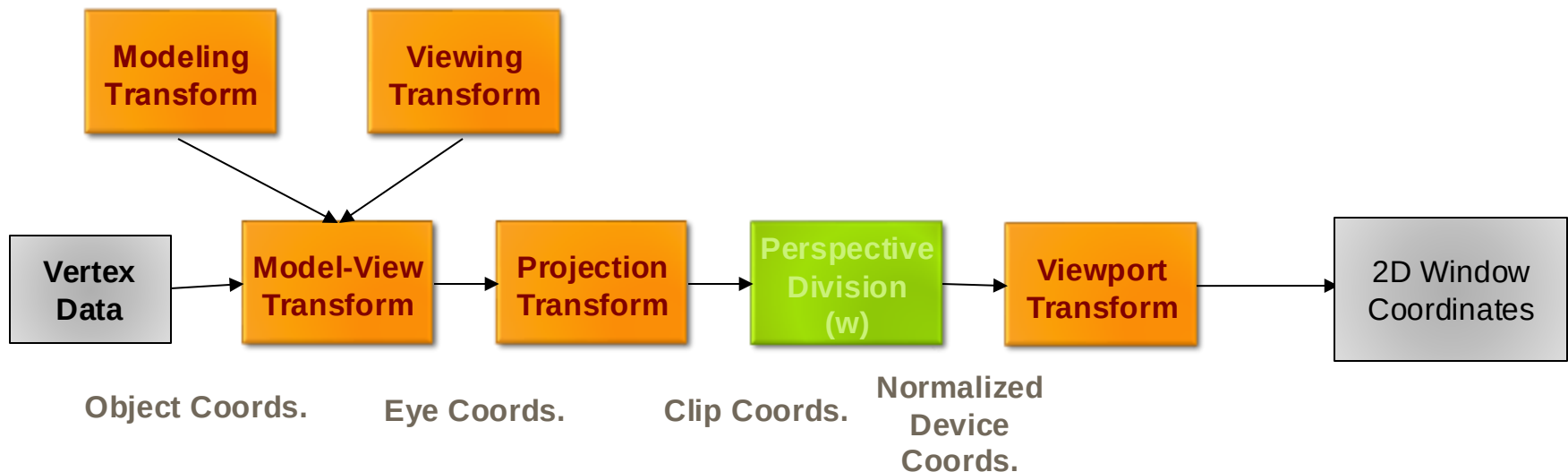
- The processing required for converting a vertex from 3D space into a 2D window coordinate is done by the transform stage of the graphics pipeline. The operations in that stage are illustrated below.
  - The orange boxes represent a matrix multiplication operation.
- Transformations take us from one “space” to another
  - All our transforms are  $4 \times 4$  matrices



# Transformations

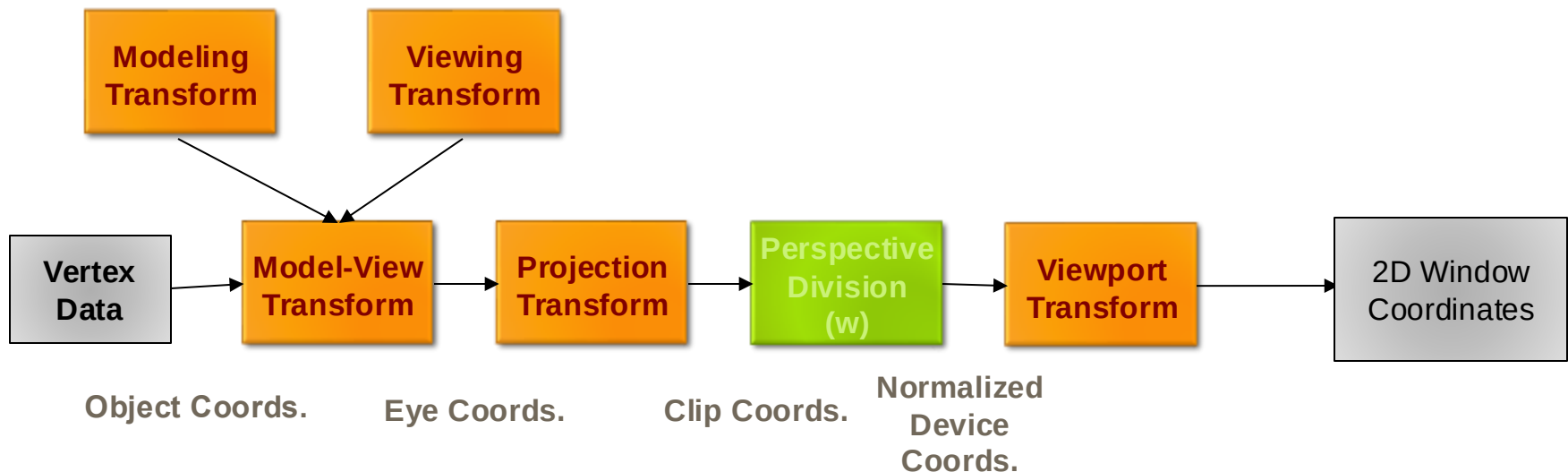
To render a 3D object, for example a chair,

1. First need to determine the vertices of the chair and then need to determine how those vertices should be grouped to form geometric primitives, and the order they're going to be sent to the graphics subsystem.
  - This process is called **modeling**. Quite often, the object is modeled in its own 3D coordinate system (called **object coordinates** or **model coordinates**).
  - We usually load this information from a model file
2. To add that object into the scene, we need to determine its **world coordinates**.
  - We do this by specifying a **modeling transformation**, which tells the system how to move from one coordinate system to another. i.e., **modeling transforms bring the object into world space**.



# Transformations

3. **Viewing transformations** specify where the viewing frustum is.
  - After model-view transformations, vertices are at the **eye coordinates (camera coordinates)** where the camera [ =eye ] works in.
4. Next, the **projection transform** is applied which maps the vertices into **clip coordinates**, where clipping occurs.
5. After clipping, we divide by the **w** value of the vertex (**perspective division**), which is modified by projection. This division operation allows the farther-objects-looking-smaller.
6. The transformed + clipped coordinates are then mapped into the window (**viewport transform**).



# Camera Analogy and Transformations

- Modeling transformations >> world coordinates
  - *moving the model in the scene*
- Viewing transformations >> eye coordinates
  - *position and orient camera in the scene*
  - Can be done by rotations and translations, or can use off-the-shelf functions such as **lookAt()**
- Projection transformations >> clip coordinates
  - *adjust the lens of the camera*
  - The **projection matrix** is used to define the view volume and to select a camera lens
- Viewport transformations >> screen coordinates
  - *enlarge or reduce the physical photograph*

# Smooth Rotation

- From a practical standpoint, we often want to use transformations to move and reorient an object smoothly
- Problem: find a sequence of model-view matrices  $\mathbf{M}_0, \mathbf{M}_1, \dots, \mathbf{M}_n$  so that when they are applied successively to one or more objects we see a smooth transition
- >> For orientating an object, we can use the fact that every rotation corresponds to part of a great circle on a sphere
  - Find the axis of rotation and angle
  - Virtual trackball (see text)

# Incremental Rotation

- Consider the two approaches
  - a) For a sequence of rotation matrices  $\mathbf{R}_0, \mathbf{R}_1, \dots, \mathbf{R}_n$ , find the Euler angles for each and use  $\mathbf{R}_i = \mathbf{R}_{iz} \mathbf{R}_{iy} \mathbf{R}_{ix}$ 
    - Not very efficient
  - b) **instead: Use the final positions to determine the axis and angle of rotation, then increment only the angle**
- Quaternions can be more efficient than either
  - But we keep those for the Advanced Computer Graphics class

# BUILDING MODELS

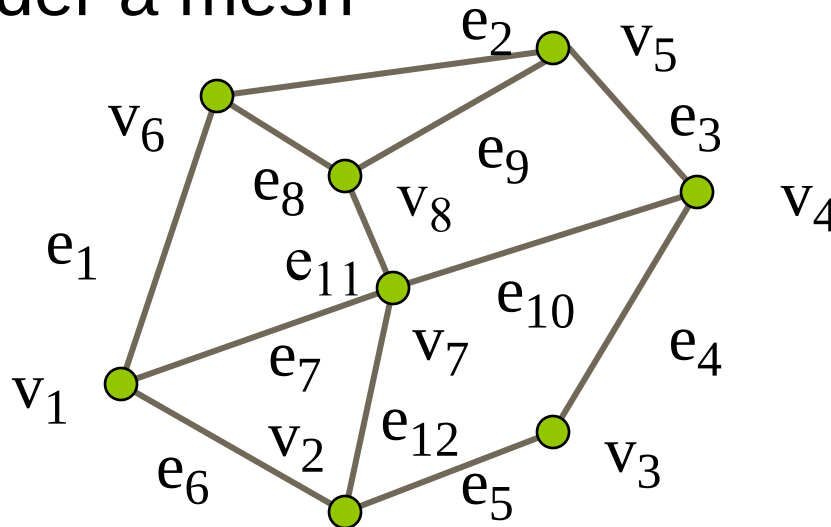
---

**Lecturer: Dr. Ufuk Çelikcan**

Based on the slides by: E. Angel and D. Shreiner

# Representing a Mesh

- Consider a mesh



- There are 8 nodes and 12 edges
  - 5 interior polygons
  - 6 interior (shared) edges
- Each vertex has a location  $v_i = (x_i \ y_i \ z_i)$

# Simple Representation

- Define each polygon by the geometric locations of its vertices

- Leads to WebGL code such as

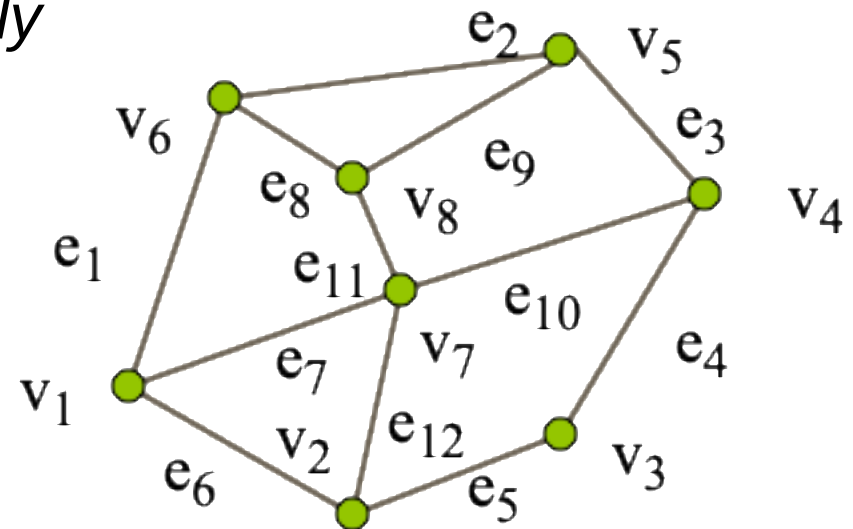
```
vertex[i] = vec3(x1, x1, x1);  
vertex[i+1] = vec3(x6, x6, x6);  
vertex[i+2] = vec3(x7, x7, x7);  
i+=3;
```

- Inefficient and unstructured

- For example: Consider moving a vertex to a new location
  - Must search for all occurrences

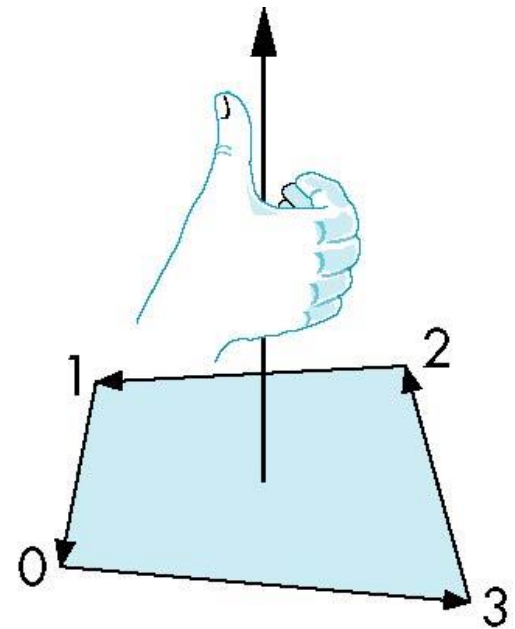
# Inward and Outward Facing Polygons

- The order  $\{v_1, v_6, v_7\}$  and  $\{v_6, v_7, v_1\}$  are equivalent in that the same polygon will be rendered by WebGL but the order  $\{v_1, v_7, v_6\}$  is different
- The first two describe *outwardly facing* polygons



# Inward and Outward Facing Polygons

- The order  $\{v_1, v_6, v_7\}$  and  $\{v_6, v_7, v_1\}$  are equivalent in that the same polygon will be rendered by WebGL but the order  $\{v_1, v_7, v_6\}$  is different
- The first two describe *outwardly facing* polygons
- Use the ***right-hand rule*** = counter-clockwise encirclement of outward-pointing normal
- WebGL can treat inward and outward facing polygons differently

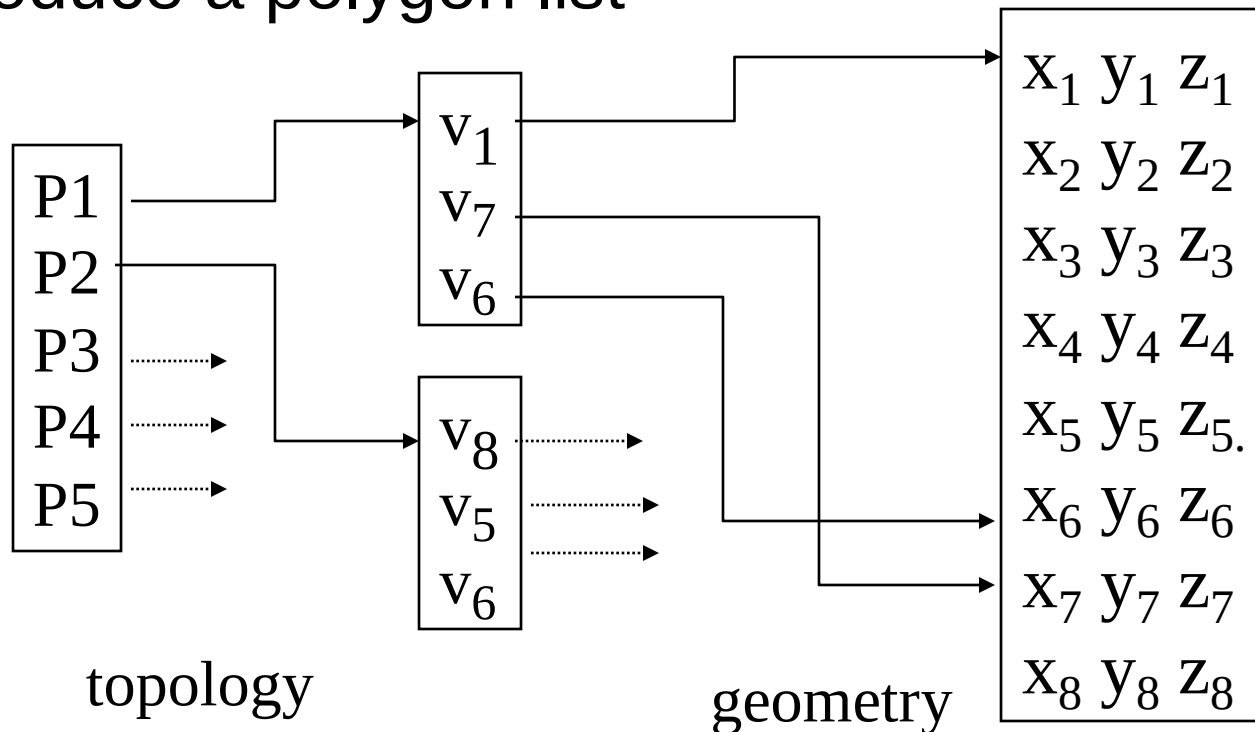


# Geometry vs Topology

- Generally it is a good idea to look for data structures that **separate the geometry from the topology**
  - **Geometry:** locations of the vertices
  - **Topology:** organization of the vertices and edges
    - Example: a polygon is an ordered list of vertices with an edge connecting successive pairs of vertices and the last to the first
  - **Topology holds even if geometry changes**

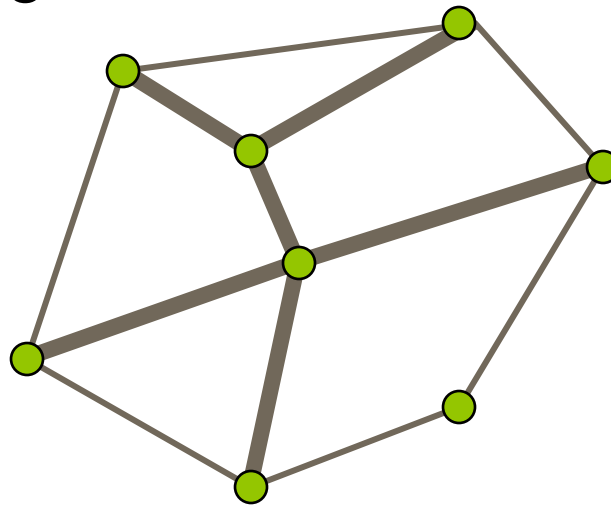
# Vertex Lists

- Put the geometry in an array
- Use pointers from the vertices into this array
- Introduce a polygon list



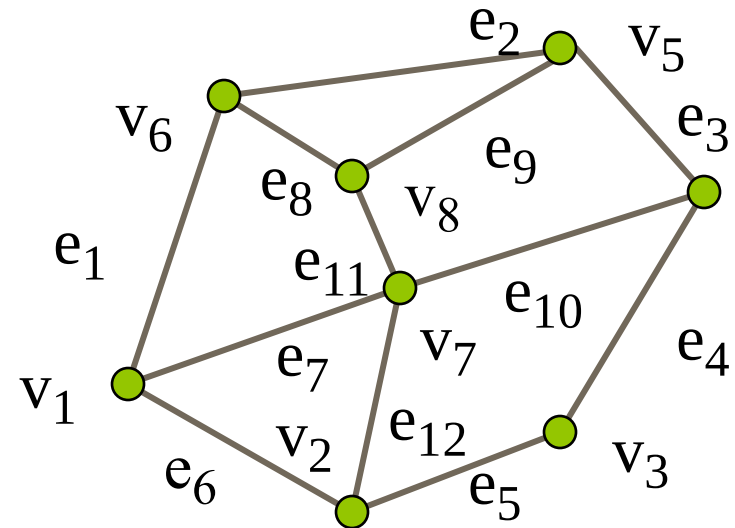
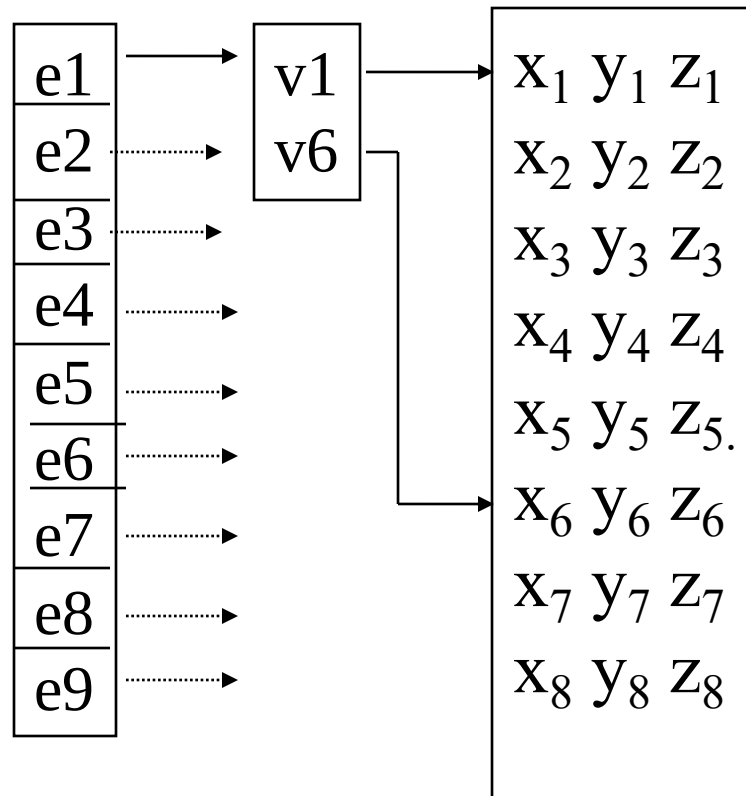
# Shared Edges

- Vertex lists will draw filled polygons correctly but if we draw the polygon by its edges, shared edges are drawn twice



- Better Alternative: Can store mesh by *edge list*

# Edge List



Note: polygons are not represented

# Modeling a Cube

Model a color cube for the rotating cube program

Define global arrays for vertices and colors

```
const vertices = [  
    -1.0,-1.0,-1.0,  
    1.0,-1.0,-1.0,  
    1.0,1.0,-1.0,  
    -1.0,1.0,-1.0,  
    -1.0,- 1.0,1.0,  
    1.0,-1.0,1.0,  
    1.0,1.0,1.0,  
    -1.0,1.0,1.0  
];
```

```
const colors = [  
    0.0,0.0,0.0,  
    1.0,0.0,0.0,  
    1.0,1.0,0.0,  
    0.0,1.0,0.0,  
    0.0,0.0,1.0,  
    1.0,0.0,1.0,  
    1.0,1.0,1.0,  
    0.0,1.0,1.0  
];
```

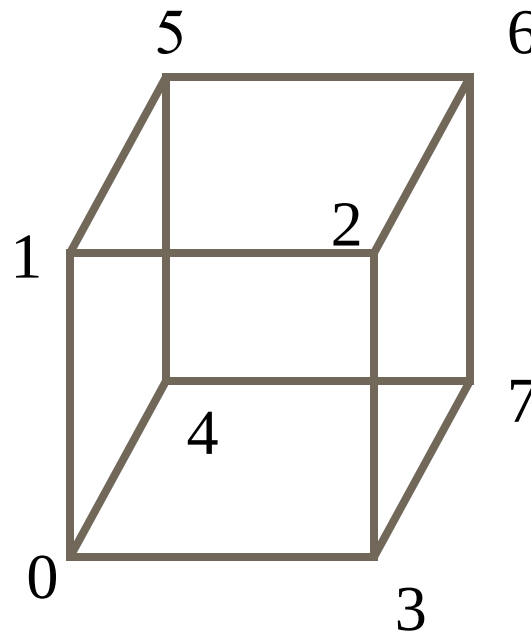
# Drawing a triangle from a list of indices

Draw a triangle from a list of indices into the array **vertices** and assign a color to each index

```
function triangle(int a, int b, int c, int
d)
{
    vcolors[i] = colors[d];
    position[i] = vertices[a];
    vcolors[i+1] = colors[d];
    position[i+1] = vertices[b];
    vcolors[i+2] = colors[d];
    position[i+2] = vertices[c];
    i+=3;
}
```

# Draw cube from faces

```
function colorcube( )  
{  
    quad(0,3,2,1) ;  
    quad(2,3,7,6) ;  
    quad(0,4,7,3) ;  
    quad(1,2,6,5) ;  
    quad(4,5,6,7) ;  
    quad(0,1,5,4) ;  
}
```



Note that vertices are ordered so that we obtain correct outward facing normals

# Efficiency

- The weakness of this approach is that we are building the model in the application and must do many function calls to draw the cube

# Vertex Arrays

- WebGL provides a facility called *vertex arrays* that allows us to store array data in the implementation
- Vertex arrays can be used for any per-vertex attributes including
  - Positions
  - Colors
  - Color indices
  - Normals
  - Texture coordinates
  - Edge flags

# Mapping indices to faces

- So instead, we can form an array of face indices

```
const cubeIndices = [0,3,2,1,2,3,7,6  
    0,4,7,3,1,2,6,5,4,5,6,7,0,1,5,4];
```

- Each successive four indices describe a face of the cube
  - But we will not pursue efficiency in our example

# Rotating Cube

- **Full example**
  - Model Colored Cube
  - Use 3 GUI buttons to change direction of rotation and another button to toggle rotation on/off
  - Use idle function to increment angle of rotation

# html code

```
<script type="text/javascript" src="Common/webgl-utils.js"></script>
<script type="text/javascript" src="Common/initShaders.js"></script>
<script type="text/javascript" src="Common/MV.js"></script>
<script type="text/javascript" src="cube.js"></script>

<body>

    <canvas id="gl-canvas" width="512" height="512">
    Oops ... your browser doesn't support the HTML5 canvas element
    </canvas>

    <br />

    <button id="xButton">Rotate X</button>
    <button id="yButton">Rotate Y</button>
    <button id="zButton">Rotate Z</button>
    <button id="tButton">Toggle Rotation</button>

</body >
```

```

"use strict";
var canvas;
var gl;
var NumVertices = 36;
var points = [], colors = [];
var xAxis = 0, yAxis = 1, zAxis = 2;
var axis = 0;
var theta = [0, 0, 0];
var thetaLoc;
var toggle = false;

var vertices = [
    vec4(-0.5, -0.5, 0.5, 1.0),
    vec4(-0.5, 0.5, 0.5, 1.0),
    vec4(0.5, 0.5, 0.5, 1.0),
    vec4(0.5, -0.5, 0.5, 1.0),
    vec4(-0.5, -0.5, -0.5, 1.0),
    vec4(-0.5, 0.5, -0.5, 1.0),
    vec4(0.5, 0.5, -0.5, 1.0),
    vec4(0.5, -0.5, -0.5, 1.0)
];

var vertexColors = [
    [0.0, 0.0, 0.0, 1.0], // black
    [1.0, 0.0, 0.0, 1.0], // red
    [1.0, 1.0, 0.0, 1.0], // yellow
    [0.0, 1.0, 0.0, 1.0], // green
    [0.0, 0.0, 1.0, 1.0], // blue
    [1.0, 0.0, 1.0, 1.0], // magenta
    [0.0, 1.0, 1.0, 1.0], // cyan
    [1.0, 1.0, 1.0, 1.0] // white
];

```

```

function colorCube() {
    quad(1, 0, 3, 2);
    quad(2, 3, 7, 6);
    quad(3, 0, 4, 7);
    quad(6, 5, 1, 2);
    quad(4, 5, 6, 7);
    quad(5, 4, 0, 1);
}

function quad(a, b, c, d) {
    // We need to partition the quad into two
    // triangles in order for WebGL to be able to
    // render it. In this case, we create two
    // triangles from the quad indices vertex color
    // assigned by the index of the vertex

    var indices = [a, b, c, a, c, d];

    for (var i = 0; i < indices.length; ++i) {
        points.push(vertices[indices[i]]);
        colors.push(vertexColors[indices[i]]);
    }
}

```

```

window.onload = function init() {
    canvas = document.getElementById("gl-canvas");

    gl = WebGLUtils.setupWebGL(canvas);
    if (!gl) { alert("WebGL isn't available"); }

    colorCube();

    gl.viewport(0, 0, canvas.width,
               canvas.height);
    gl.clearColor(1.0, 1.0, 1.0, 1.0);

    gl.enable(gl.DEPTH_TEST);

    // Load shaders and initialize attribute
    buffers
    var program = initShaders(gl, "vertex-shader",
                              "fragment-shader");
    gl.useProgram(program);

    var cBuffer = gl.createBuffer();
    gl.bindBuffer(gl.ARRAY_BUFFER, cBuffer);
    gl.bufferData(gl.ARRAY_BUFFER,
                  flatten(colors), gl.STATIC_DRAW);

    var vColor = gl.getAttribLocation(program,
                                       "vColor");
    gl.vertexAttribPointer(vColor, 4, gl.FLOAT,
                           false, 0, 0);
    gl.enableVertexAttribArray(vColor);

```



*Function continues on the next column*

```

    var vBuffer = gl.createBuffer();
    gl.bindBuffer(gl.ARRAY_BUFFER, vBuffer);
    gl.bufferData(gl.ARRAY_BUFFER,
                  flatten(points), gl.STATIC_DRAW);

    var vPosition = gl.getAttribLocation(program,
                                          "vPosition");
    gl.vertexAttribPointer(vPosition, 4, gl.FLOAT,
                           false, 0, 0);
    gl.enableVertexAttribArray(vPosition);

    thetaLoc =
        gl.getUniformLocation(program, "theta");

    //event listeners for buttons
    document.getElementById("xButton").onclick =
        function () { axis = xAxis; };
    document.getElementById("yButton").onclick =
        function () { axis = yAxis; };
    document.getElementById("zButton").onclick =
        function () { axis = zAxis; };
    document.getElementById("tButton").onclick =
        function () { toggle = !toggle; };

    render();
}

function render() {
    gl.clear(gl.COLOR_BUFFER_BIT |
            gl.DEPTH_BUFFER_BIT);

    if (toggle) theta[axis] += 2.0;
    gl.uniform3fv(thetaLoc, theta);
    gl.drawArrays(gl.TRIANGLES, 0, NumVertices);

    requestAnimationFrame(render);
}

```

# Vertex Shader

```

attribute vec4 vPosition;
attribute vec4 vColor;
varying vec4 fColor;
uniform vec3 theta;

void main()
{
    // Compute the sines and cosines of theta for each of
    // the three axes in one computation.
    vec3 angles = radians(theta);
    vec3 c = cos(angles);
    vec3 s = sin(angles);

    // Remember: these matrices are column-major
    mat4 rx = mat4(1.0, 0.0, 0.0, 0.0,
                  0.0, c.x, s.x, 0.0,
                  0.0, -s.x, c.x, 0.0,
                  0.0, 0.0, 0.0, 1.0);

    mat4 ry = mat4(c.y, 0.0, -s.y, 0.0,
                  0.0, 1.0, 0.0, 0.0,
                  s.y, 0.0, c.y, 0.0,
                  0.0, 0.0, 0.0, 1.0);

    mat4 rz = mat4(c.z, s.z, 0.0, 0.0,
                  -s.z, c.z, 0.0, 0.0,
                  0.0, 0.0, 1.0, 0.0,
                  0.0, 0.0, 0.0, 1.0);

    fColor = vColor;
    gl_Position = rz * ry * rx * vPosition;
    gl_Position.z = -gl_Position.z;
}

```

$$\leftarrow \mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\leftarrow \mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\leftarrow \mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

# Fragment Shader

```
precision mediump float;
```

```
in vec4 fColor;
```

```
out vec4 outColor;
```

```
void main()
```

```
{
```

```
    outColor = fColor;
```

```
}
```